

Министерство цифрового развития, связи и массовых коммуникаций РФ
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра вычислительных систем
Допустить к защите
Зав. кафедрой к.т.н., доцент
_____ Перышкова Е.Н.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Разработка системы управления дроном с
применением нейронной сети

Пояснительная записка

Студент Молодцов И.С.

Институт ИВТ Группа ИВ-922

Руководитель Гонцова А. В.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1) Инструментарий	6
1.1 Библиотека TensorFlow	6
1.2 Библиотека MAVROS.....	12
1.3 Набор для полетов беспилотников Pixhawk и PX4.....	12
1.4 Расширенный алгоритм фильтра Калмана	13
2) Архитектура Нейронной сети.....	16
2.1 Предлагаемая система.....	16
2.2 Предварительная обработка данных	17
2.2.1 Разная частота обновления различных зарегистрированных сообщений.....	17
2.2.2 Отображение данных в окне	18
2.2.3 Различие	18
2.2.4 Обрезка времени заземления	20
2.2.5 Ручная очистка набора данных	22
2.3 Проектирование сети и настройка гиперпараметров	23
2.3.1 Общие требования.....	23
2.3.2 Функция потерь	23
2.4 Настройка гиперпараметров	25
2.4.1 Количество Узлов	25
2.4.2 Тип повторяющихся ячеек и функции активации	25
2.4.3 Размер окна	25
2.4.4 Скорость обучения	25
2.4.5 Размер партии	26
2.4.6 Регуляризация	26
2.4.7 Время обучения и количество эпох	26
2.4.8 Окончательный дизайн	27
3) Обучение сети.....	28
3.1.1 Результаты и обсуждение	29
3.1.2 Производительность набора для тренировки.....	29
3.1.3 Производительность набора для проверки.....	29
4) Тестирование	34
4.1 Предсказания скорости	34

4.2 Предсказания отношения.....	36
ЗАКЛЮЧЕНИЕ.....	37
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	38
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	39
ПРИЛОЖЕНИЕ А1	41
ПРИЛОЖЕНИЕ А2	42
ПРИЛОЖЕНИЕ Б1	43
ПРИЛОЖЕНИЕ Б2	44
ПРИЛОЖЕНИЕ В.....	45
ПРИЛОЖЕНИЕ Г	46

ВВЕДЕНИЕ

Автономный полет беспилотника требует непрерывной и надежной оценки ориентации, скорости и положения беспилотника. Обычно эти условия оцениваются путем применения расширенного фильтра Калмана (EKF) к показаниям акселерометра, гироскопа, барометра, магнитометра и GPS. Эти условия оцениваются путем применения расширенного фильтра Калмана (EKF) к показаниям барометра, магнитометра и GPS; потеря сигнала GPS быстро ухудшает оценки положения и скорости. В частности, оценки положения и скорости быстро ухудшаются при использовании недорогих инерциальных датчиков. В данной работе предлагается следующий метод оценки. Для надежного определения положения и скорости в отсутствие сигналов GPS предлагается метод оценки на основе рекуррентной нейронной сети (RNC). Предлагается метод оценки на основе рекуррентной нейронной сети (RNC) для получения надежных оценок положения и скорости беспилотника в отсутствие сигналов GPS. RNC обучается на общем наборе данных, собранных с помощью Pixhawk. Этот недорогой коммерческий автопилот записывает измерения датчиков (сетевые входы). Rraw записывает измерения датчиков (входы сети) и соответствующие предсказания EKF (выходы "наземной истины"). Этот набор данных. Набор данных состоит из 548 различных журналов полетов с продолжительностью полета от 4 до 32 минут. Для целей обучения было использовано 465 полетов общей продолжительностью 45 часов. Всего было использовано 45 часов полетов. Оставшиеся 83 полета (общей продолжительностью 8 часов) были сохранены для валидации; ошибки из-за облета Максимальная абсолютная разница (MPE) в трехмерном положении между прогнозом (без GPS) и наземной истиной (EKF с GPS). Тематическое разбиение валидационной сети составляет 35 миллионов; максимальное значение, зарегистрированное в течение 5 минут полета, составляет 2,7 м. В 90 процентах испытательных полетов значение MPEI было ниже 166 м. Сеть была проверена экспериментально и работала в режиме реального времени.

Многие приложения требуют точного определения местоположения без использования GPS. Существует множество приложений, требующих

точного позиционирования без использования GPS. Примерами могут служить навигация роботов внутри помещений на складах и в гаражах, подводные работы AUV (автономных подводных роботов), а также передвижение без водителя и беспилотных автомобилей в зонах без GPS. Сюда входят автомобили без водителя и беспилотники, передвигающиеся в туннелях, под мостами и в плотной городской среде, а также автономные подводные роботы (AUV), работающие под водой. Сюда входят беспилотные транспортные средства и беспилотники, передвигающиеся под туннелями и мостами, а также в плотной городской среде. Для решения проблемы навигации в условиях отсутствия GPS был применен ряд подходов. Для решения проблем навигации в условиях недоступности GPS применяется ряд подходов. В роботизированных и беспилотных транспортных средствах В роботизированных и беспилотных автомобилях инерциальные датчики часто дополняются камерами, лазерными сканерами, радарами и датчиками автомобиля. В диагностике транспортных средств (OBD). Однако эти датчики не только дороги. Однако эти датчики не только дороги для правильной работы, но и накладывают ограничения на окружающую среду или поведение автомобиля. Они ограничены окружающей средой или поведением автомобиля и работают некорректно. До появления GPS в качестве стандартного метода определения местоположения самолетов, инерциальные навигационные системы Инерциальная навигационная система (ИНС). ИНС оценивает положение по ускорению и угловой скорости с помощью хорошо откалиброванных инерциальных датчиков высокого порядка и передовых навигационных алгоритмов. Она оценивает положение по ускорению и угловой скорости. Однако стоимость таких подходов не может быть оправдана в коммерческом контексте. Недавней тенденцией является использование технологий искусственного интеллекта (ИИ) для помощи ИНС. Окна оценки и измерения акселерометра были использованы для оценки ошибок в положении и скорости, оцененных ИНС. Кроме того, при использовании в комбинации с EKF. EKF могут использоваться в сочетании с EKF, использующими акселерометры и гироскопы (далее - гироскопы) для оценки положения. Они также могут быть использованы в сочетании с EKF для оценки положения на основе измерений

акселерометров, гироскопов (совместно называемых инерциальными измерительными блоками IMUs) и магнитометров. Также широко используются версии многослойных перцептронов. Они широко используются для оценки приращений скорости и положения по азимуту и для оценки ошибок по азимуту и скорости. Он также оценивает ошибки в направлении и скорости из-за изменений от одного временного шага к другому. Он использовался для оценки. Он также использовался для оценки изменений широты и долготы по измерениям скорости IMU и INS. Она также использовалась для оценки изменений широты и долготы на основе измерений INS и скоростей INS. Недавно были использованы рекуррентные нейронные сети (RНС) из-за их превосходства над изменениями временных рядов. Благодаря своему превосходству, они используются для решения задач, связанных с временными рядами. RНС обучаются оценивать ошибку положения и скорости, используя скорость и изменения курса ИНС в качестве входных данных, а сети с долговременной кратковременной памятью (LSTM), которые являются модифицированными RНС, могут также использовать историю усиления Калмана. История коэффициента усиления фильтра Калмана может быть использована для оценки ошибок скорости и положения ИНС. Таким образом, можно оценить ошибки скорости и положения. Другие типы нейронных сетей включают нейронные сети с входной задержкой (IDNN), ансамблевое обучение, нелинейная авторегрессия с экзогенным входом (NARX), нечеткие Логические системы вывода (FIS) также были реализованы с многообещающими результатами.

1 Инструментарий

1.1 Библиотека TensorFlow

TensorFlow, созданная командой Google Brain и впервые выпущенная для широкой публики в 2015 году, представляет собой библиотеку с открытым исходным кодом для численных вычислений и крупномасштабного машинного обучения. TensorFlow объединяет множество моделей и алгоритмов машинного и глубокого обучения (также известных как нейронные сети) и делает их полезными с помощью общих программных метафор.

Он использует Python или JavaScript для предоставления удобного интерфейсного API для создания приложений при выполнении этих приложений на высокопроизводительном C++.

Приложения TensorFlow можно запускать практически на любой удобной цели: локальный компьютер, кластер в облаке, устройства iOS и Android, центральные или графические процессоры. Если вы используете собственное облако Google, вы можете запустить TensorFlow на собственном кремнии Google TensorFlow Processing Unit (TPU) для дальнейшего ускорения. Однако результирующие модели, созданные TensorFlow, могут быть развернуты практически на любом устройстве, где они будут использоваться для прогнозирования.

TensorFlow 2.0, выпущенный в октябре 2019 года, во многом изменил структуру на основе отзывов пользователей, чтобы упростить работу (например, за счет использования относительно простого API-интерфейса Keras для обучения модели) и сделать ее более производительной. Распределенное обучение проще проводить благодаря новому API, а поддержка TensorFlow Lite позволяет развертывать модели на самых разных платформах. Однако код, написанный для более ранних версий TensorFlow, необходимо переписать — иногда незначительно, иногда значительно — чтобы максимально использовать преимущества новых функций TensorFlow 2.0.

Приложения глубокого обучения очень сложны, а процесс обучения требует большого количества вычислений. Это занимает много времени из-за

большого размера данных и включает в себя несколько итерационных процессов, математические

вычисления, умножение матриц и так далее. Если вы выполняете эти действия на обычном центральном процессоре (ЦП), обычно это занимает гораздо больше времени.

Графические процессоры (GPU) популярны в контексте игр, где вам нужно, чтобы экран и изображение имели высокое разрешение. Графические процессоры изначально были разработаны для этой цели. Однако они также используются для разработки приложений глубокого обучения.

Одним из основных преимуществ TensorFlow является то, что он поддерживает как графические процессоры, так и процессоры. Он также имеет более быстрое время компиляции, чем другие библиотеки глубокого обучения, такие как Keras и Torch.

Архитектура Tensorflow работает в три важных этапа:

Предварительная обработка данных — структурирование данных и приведение их к одному предельному значению

- Построение модели - построить модель для данных
- Обучение и оценка модели — используйте данные для обучения модели и проверки ее с неизвестными данными.

Глубокое обучение — это разновидность машинного обучения, и оно работает по структуре и функциям аналогично человеческому мозгу. Он учится на неструктурированных данных и использует сложные алгоритмы для обучения нейронной сети.

Входной слой принимает большие объемы данных в качестве входных данных для построения нейронной сети. Данные могут быть в виде текста, изображения, аудио и т. д.

Скрытый слой обрабатывает данные, выполняя сложные вычисления и извлекая признаки. В рамках обучения эти слои имеют веса и смещения, которые постоянно обновляются до завершения процесса обучения. Каждый нейрон имеет несколько весов и одно смещение. После вычисления значения передаются на выходной слой.

Выходной слой генерирует прогнозируемый результат, применяя подходящие функции активации. Вывод может быть в виде числовых или категориальных значений.

Например, если это приложение для классификации изображений, оно сообщает, к какому классу может принадлежать конкретное изображение. Входными данными могут быть несколько изображений, например кошек и собак. Вывод может быть в виде двоичной классификации, например, число ноль для собаки и число один для кошки.

Сеть может быть расширена несколькими нейронами на стороне вывода, чтобы иметь гораздо больше классов. Его также можно использовать для задач регрессии и временных рядов.

Есть несколько предпосылок, необходимых для разработки приложения глубокого обучения. Вам необходимо хорошее знание Python, но также полезно знать другие языки программирования, такие как R, Java или C++.

TensorFlow — это библиотека с открытым исходным кодом, разработанная Google в первую очередь для приложений глубокого обучения. Он также поддерживает традиционное машинное обучение. Изначально TensorFlow разрабатывался для больших числовых вычислений без учета глубокого обучения. Однако он оказался очень полезным и для разработки глубокого обучения, и поэтому Google открыл его исходный код.

TensorFlow принимает данные в виде многомерных массивов более высоких размерностей, называемых тензорами. Многомерные массивы очень удобны при обработке больших объемов данных.

TensorFlow работает на основе графов потоков данных, которые имеют узлы и ребра. Поскольку механизм выполнения представлен в виде графов, гораздо проще выполнять код TensorFlow распределенным образом в кластере компьютеров с использованием графических процессоров.

Впервые TensorFlow был обнародован в 2015 году, а первая стабильная версия появилась 11 февраля 2017 года. Он создан и поддерживается Google. С тех пор он стал одним из самых популярных фреймворков для проектов глубокого и машинного обучения. Он имеет обширную библиотеку для

крупномасштабного машинного обучения и численных вычислений. Вот еще несколько вех TensorFlow:

В декабре 2017 года выпущен KubeFlow для эксплуатации и развертывания TensorFlow на Kubernetes. В марте 2018 года был выпущен TensorFlow 1.0 для машинного обучения в JavaScript.

В январе 2019 года выходит TensorFlow 2.0, который добавляет в TensorFlow ряд компонентов. В мае 2019 года был выпущен TensorFlow Graphics для глубокого обучения компьютерной графике.

TensorFlow предлагает API C++ и Python. До разработки библиотек механизм кодирования для машинного обучения и глубокого обучения был намного сложнее. Эта библиотека предоставляет высокоуровневый API, и для подготовки нейронной сети, настройки или программирования нейрона не требуется сложного кодирования. Библиотека выполняет все эти задачи. TensorFlow также имеет интеграцию с Java и R.

TensorFlow поддерживает вычислительные устройства как с процессорами, так и с графическими процессорами. Приложения глубокого обучения очень сложны, а процесс обучения требует большого количества вычислений. Это занимает много времени из-за большого размера данных и включает в себя несколько итерационных процессов, математические вычисления, умножение матриц и так далее. Если вы выполняете эти действия на обычном центральном процессоре (ЦП), обычно это занимает гораздо больше времени.

Графические процессоры (GPU) популярны в контексте игр, где вам нужно, чтобы экран и изображение имели высокое разрешение. Графические процессоры изначально были разработаны для этой цели. Однако они также используются для разработки приложений глубокого обучения.

Одним из основных преимуществ TensorFlow является то, что он поддерживает как графические процессоры, так и процессоры. Он также имеет более быстрое время компиляции, чем другие библиотеки глубокого обучения, такие как Keras и Torch.

TensorFlow позволяет создавать графы потоков данных, которые описывают, как данные перемещаются по графу. Граф состоит из узлов, которые представляют математическую операцию. Соединение или ребро

между узлами представляет собой многомерный массив данных. Он принимает входные данные в виде многомерного массива, в котором вы можете построить блок-схему операций, которые можно выполнять с этими входными данными.

Архитектура Tensorflow работает в три важных этапа:

- Предварительная обработка данных — структурирование данных и приведение их к одному предельному значению
- Построение модели - построить модель для данных
- Обучение и оценка модели — используйте данные для обучения модели и проверки ее с неизвестными данными.

Требования TensorFlow можно разделить на этап разработки (обучение модели) и этап запуска (запуск модели на разных платформах). Модель можно обучать и использовать как на GPU, так и на CPU. После того, как модель была обучена, вы можете запустить ее на:

- Рабочий стол (Linux, Windows, macOS)
- Мобильные устройства (iOS и Android)
- Облако как веб-сервис Тензор

Tensor формирует основную структуру TensorFlow. Все вычисления в TensorFlow включают тензоры. Это матрица n измерений, которая представляет несколько типов данных. Тензор может быть результатом вычислений или исходить из входных данных.

Графики описывают все операции, которые происходят во время обучения. Каждая операция называется операционным узлом и связана с другой. На графике показаны операционные узлы и связи между узлами, но не отображаются значения.

Тензор — это обобщение векторов и матриц потенциально более высоких размерностей. Массивы данных с различными размерами и рангами, которые подаются в качестве входных данных для нейронной сети, называются тензорами.

Для глубокого обучения, особенно в процессе обучения, будут использоваться большие объемы данных, которые существуют в очень сложном формате. Это помогает, когда вы можете поместить, использовать

или сохранить его компактным способом, который обеспечивают тензоры, даже если они появляются в многомерных массивах.

Размерность — это размер элементов массива. Ниже вы можете ознакомиться с различными типами размеров

Тензорные ранги — это количество измерений, используемых для представления данных. Например:

Ранг 0 — когда есть только один элемент. также называется это скаляром. Пример: $c = [2000]$

Ранг 1. В основном это относится к одномерному массиву, называемому вектором. Пример: $v = [10, 11, 12]$

Ранг 2 — традиционно известен как двумерный массив или матрица. Пример: $m = [1,2,3],[4,5,6]$

Ранг 3 — относится к многомерному массиву, обычно называемому тензором. Пример: $t = [[[1],[2],[3]],[[4],[5],[6]],[[7],[8],[9]]]$

Тогда рангов может быть четыре или пять и так далее.

Граф потока данных — еще одна важная тема в статье «Что такое TensorFlow». Это обсуждается в следующем разделе.

Когда есть данные, хранящиеся в тензорах, необходимо выполнить вычисления, которые происходят в виде графиков.

В отличие от традиционного программирования, где написанный код выполняется последовательно, здесь строятся графы потоков данных, состоящие из узлов. Затем графики выполняются в виде сеанса. Когда создаётся объект TensorFlow, будет график по умолчанию. В более продвинутом программировании у вас может быть несколько графиков вместо графика по умолчанию. Вы также можете создать свой собственный график. Граф выполняется и обрабатывает все поступающие данные. Все внешние данные передаются в виде заполнителей, переменных и констант.

Когда у вас есть граф, выполнение может быть включено либо на обычных процессорах, либо на графических процессорах, либо распределено между несколькими из них, чтобы обработка стала намного быстрее. Поскольку обучение моделей глубокому обучению занимает очень много времени из-за большого объема данных, использование TensorFlow

значительно упрощает написание кода для графических или центральных процессоров, а затем выполняет его распределенным образом.

1.2 Библиотека MAVROS

ROS предоставляет библиотеки, инструменты, аппаратную абстракцию, драйверы устройств, визуализаторы, передачу сообщений, управление пакетами и многое другое, чтобы помочь разработчикам программного обеспечения создавать приложения для роботов. В будущем ожидается, что ROS будет заменен на ROS2.

MAVROS — это пакет ROS, который может преобразовывать темы ROS в сообщения MAVLink, позволяя транспортным средствам ArduPilot взаимодействовать с ROS.

1.3 Набор для полетов беспилотников Pixhawk и PX4

Pixhawk 4 — это усовершенствованный автопилот, разработанный и созданный в сотрудничестве с Holybro и командой PX4. Он оптимизирован для запуска PX4 v1.7 и более поздних версий и подходит для академических и коммерческих разработчиков.

В дополнение к низкоуровневым драйверам датчиков, контурам управления и алгоритму планирования, автопилот PX4 имеет программу интеграции INS/GPS, называемую EKF2, как часть своей библиотеки оценки и управления (ECL). EKF2 объединяет измерения от ИИУ, барометра, магнитометра и GPS для оценки состояний дрона, а именно ориентации, представленной в кватернионах, скорости и положения в локальной системе отсчета (северо-восток-вниз). Оценки EKF2 также записываются на SD-карту.

PX4 ведет базу данных журналов полетов, куда пользователи по всему миру загружают фактические данные о полетах, собранные в ходе реальных полетов. Данные, используемые в этой работе, загружены из базы данных PX4. База данных содержит тысячи журналов, собранных с использованием различных версий Pixhawk и с использованием различных дронов. Модели датчиков, используемых на плате Pixhawk4, перечислены в таблице 1.1.

Таблица 1.1 – Датчики Pixhawk4

Сенсор	Модель
Акселерометр	ICM-20689
Гироскоп	BMI055
Магнитометр	IST8310
Барометр	MS5611

Журналы, загруженные из базы данных PX4, содержат необработанные измерения датчиков (ИИУ, барометр и магнитометр); они обрабатываются как входные данные сети (функции). Журналы также содержат состояния полета, оцененные с помощью EKF2 (ориентация, скорость и положение); они обрабатываются как выходные данные сети (метки). Другие данные о работоспособности оборудования также регистрируются, и они используются для удаления поврежденных журналов.

Записи были собраны с помощью различных типов носителей, в основном квадрокоптеров, но также использовались некоторые фиксированные крылья и аппараты вертикального взлета и посадки.

1.4 Расширенный алгоритм фильтра Калмана

Алгоритм EKF2 PX4 Вместо того, чтобы пытаться оценить ориентацию кватерниона напрямую, он оценивает вектор ошибки вращения и применяет поправку к кватерниону из уравнений инерциальной навигации. Это лучше, когда есть большие угловые ошибки, поскольку это позволяет избежать ошибок, связанных с линеаризацией кватернионов при больших изменениях угла.. На этапе прогнозирования дельта-угол и дельта-скорость получаются путем численного интегрирования угловых скоростей a_x ; a_y ; a_z , измеренных гироскопами на раме транспортного средства, и ускорений, измеренных акселерометрами на той же раме. Эти члены представляют все источники инерциальной навигационной ошибки и обновляются на каждой итерации как часть вектора состояния EKF.

Полученные дельта-углы корректируются с учетом влияния скорости вращения Земли, которая является функцией широты. Скорректированные дельта-углы преобразуются в дельта-кватернионы, которые объединяются с предыдущими кватернионами q_1 , q_2 , q_3 и q_4 для расчета нового положения автомобиля.

Это положение используется для преобразования из системы координат тела в локальную систему координат. Затем добавляются гравитационные эффекты, зависящие от широты. Этот процесс показан в Приложении A1 и A2.

Поскольку численное интегрирование выполняется три раза, а смещение для коррекции не совсем корректно, эти расчетные значения быстро отклоняются от фактических значений состояния. На рисунке 1.1 показаны оценки полета, когда GPS не используется для компенсации оценок ИИУ.

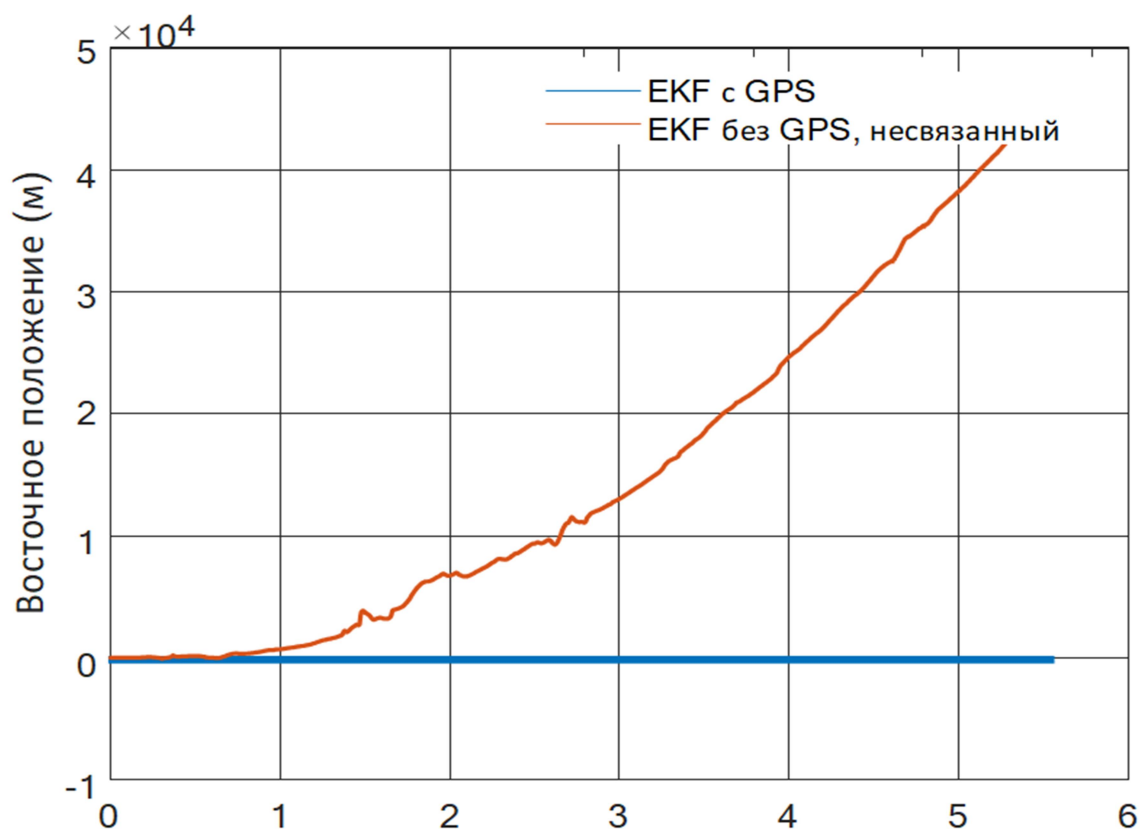


Рисунок 1.1 – Предсказание восточного положения EKF 2 при отсутствии помощи GPS

Вторым шагом в EKF2 является коррекция. Всякий раз, когда доступны измерения с другого датчика, прогнозы, полученные на первом этапе, корректируются. Барометр используется для коррекции нисходящей составляющей положения, в то время как GPS корректирует две другие составляющие положения и три составляющие скорости. Магнитометр помогает скорректировать положение. В приложении B1 показана схема архитектуры прогнозирования и коррекции EKF.

Датчики, отличные от ИИУ, также используются для коррекции отклонений ИИУ на этапе коррекции, чтобы обеспечить лучшие прогнозы ИИУ на следующем временном шаге. Коррекция смещения ИИУ осуществляется путем добавления смещений в дельта-углах и дельта-скоростях к вектору состояния EKF.

1.5

2 Архитектура Нейронной сети

2.1 Предлагаемая система

Основная идея заключается в том, чтобы заменить процесс, показанный в Приложении А1 и А2, нейронной сетью ИНС в Приложении В2. Работа сети в основном заключается в изучении моделей ошибок датчиков, поворотов измерений и интеграции. ИНС здесь можно рассматривать как функцию подгонки сложной кривой. При подгонке кривых строится полиномиальная функция, представляющая связь между независимой переменной, такой как время, и зависимой переменной, такой как цена товара. Коэффициенты этого полинома рассчитываются на основе набора пар "вход-выход". После расчета коэффициентов полинома в него могут быть вставлены новые значения времени, например, для прогнозирования будущих стоимостей. Затем рассчитанная подгоночная функция проверяется на достоверность при новых входных данных с известными выходными данными. Этот тест подтверждает точность ранее ненаблюдаемых данных (валидационного набора) на момент подгонки. ИНС имеет ту же концепцию, но сочетание нелинейной функции и набора параметров, называемых весами, позволяет подгонять более сложные взаимосвязи. В данном случае независимыми переменными (входные ИНС) являются измерения ИИУ, барометра и магнитометра. Зависимыми переменными (выходными ИНС) являются изменения в положении и скорости.

Архитектура сети, показанная в Приложении В, называется плотной архитектурой. Лучшей альтернативой, где входные или выходные данные представлены в виде серии данных, является рекуррентная нейронная сеть (РНС). РНС подходят для серий любой длины и популярны в приложениях распознавания речи и прогнозирования временных рядов. РНС можно использовать для передачи информации из памяти от более ранних точек последовательности к более поздним точкам последовательности и передачи информации из памяти для последующих предсказаний. Используемая архитектура представляет собой один плотный слой, за которым следует рекуррентный слой.

2.2 Предварительная обработка данных

2.2.1 Разная частота обновления различных зарегистрированных сообщений

К необработанным измерениям датчиков, передаваемым в сеть, фильтрация не применяется. Все упомянутые здесь этапы предварительной обработки используются для управления скоростью ввода, удаления поврежденных данных из обучающего набора или разделения входных данных на сегменты.

В файле журнала полетов, созданном PX4 stack, разные данные датчиков хранятся в разных сообщениях. Частота регистрируемых данных не одинакова во всех сообщениях; разные частоты показаны в таблице 2.1. Это означает, что между двумя последовательными выходами, ожидаемыми от сети, поступает несколько входных данных. Кроме того, размер входных данных отличается от шага к шагу; например, на одном шаге доступны только измерения ИИУ, но на следующем шаге могут быть доступны только данные барометра и так далее. Чтобы решить эту проблему, данные датчика усредняются между каждыми двумя последовательными зарегистрированными выходами EKF (приложении В). Это усреднение объединяет входную и выходную частоты до 5 Гц.

Таблица 2.1 – Различные показатели работы датчиков в файле журнала

Устройство	Модель	Частота обновления
Акселерометр и Гироскоп	Угловые скорости $\omega_x, \omega_y, \omega_z$ и линейные ускорения a_x, a_y, a_z	84 Hz
Барометр	Температура T и Высота h	67 Hz
Магнитометр	Компоненты магнитного поля M_x, M_y, M_z	45 Hz
EKF	Предполагаемые состояния, Ориентация q_1, q_2, q_3, q_4 , Скорость V_N, V_E, V_D и позиция P_N, P_E, P_D	5 Hz

2.2.2 Отображение данных в окне

Причина использования РНС с массивами заключается в исследовании зависимостей в массивах данных. Длина последовательности, отправляемой в сеть за один раз, называется размером окна. В этой задаче длина окна определяется количеством временных шагов, зная, что каждый временной шаг равен 0,2 секунды. Оптимальный размер окна зависит от задачи и

считается конструктивным параметром и обычно выбирается путем испытаний. Большие окна требуют больше времени как для обучения, так и для вывода, а окна меньшего размера могут не охватывать достаточное количество зависимостей.

2.2.3 Различие

Инерциальные датчики измеряют скорость, поэтому их можно использовать только для вычисления приращений в состояниях полета (положение и скорость). Затем абсолютные значения состояний вычисляются путем суммирования этих приращений к начальным условиям. Вот почему лучше, чтобы сеть предсказывала приросты и накапливала прогнозы позже. Таким образом, при предварительной обработке разность применяется к меткам (положение, скорость и ориентация) и к входной высоте барометра. Вычисление различий производится путем вычитания каждой точки в последовательности из следующей.

Еще одним преимуществом этого подхода является то, что на предсказание временного шага не влияет неправильное предсказание предыдущего шага, а только восстановленный путь. В результате ошибка в предсказанном состоянии локализована, и в полете существует постоянное отклонение между фактическим и предсказанным состоянием, но эта ошибка не увеличивается со временем. На рисунках 2.1 и 2.2 показано, как неправильные прогнозы приводят к ограниченной ошибке, которая не увеличивается со временем. Это возможно только потому, что прогнозируются изменения в состояниях, а не сами состояния. Смещение увеличивается, когда совершается несколько последовательных ошибок в одном и том же направлении, и уменьшается, когда совершаются ошибки противоположного направления.

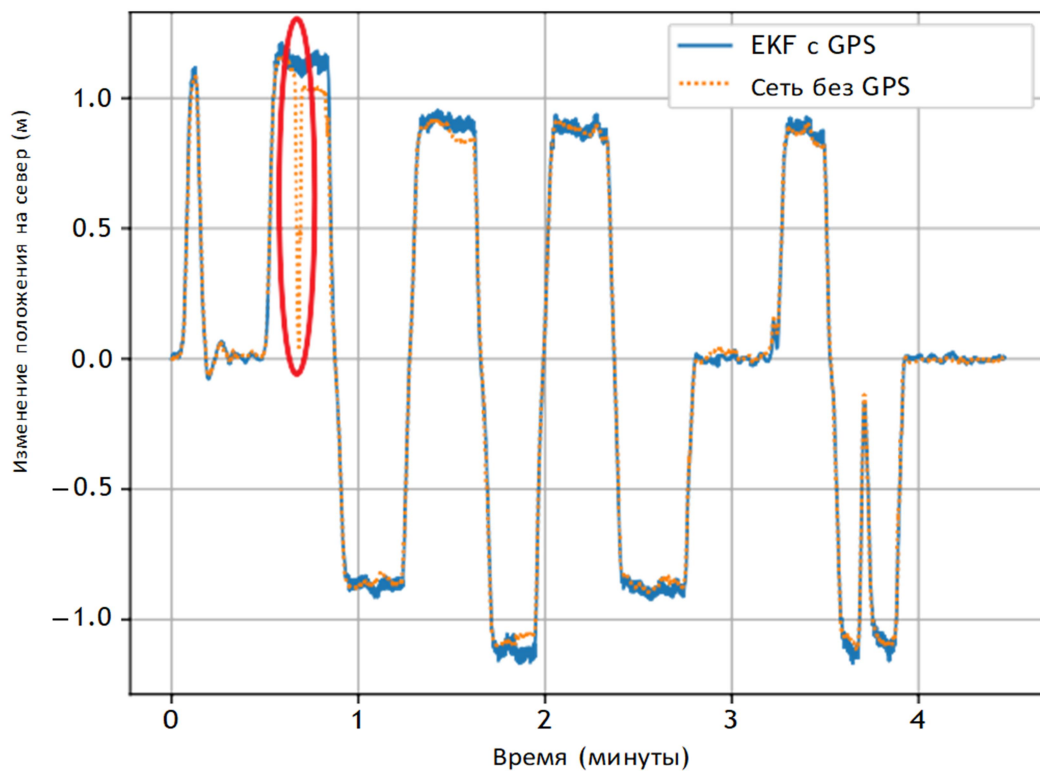


Рисунок 2.1 – Изменения в состоянии, предсказанные сетью, в отличие от истинных изменений

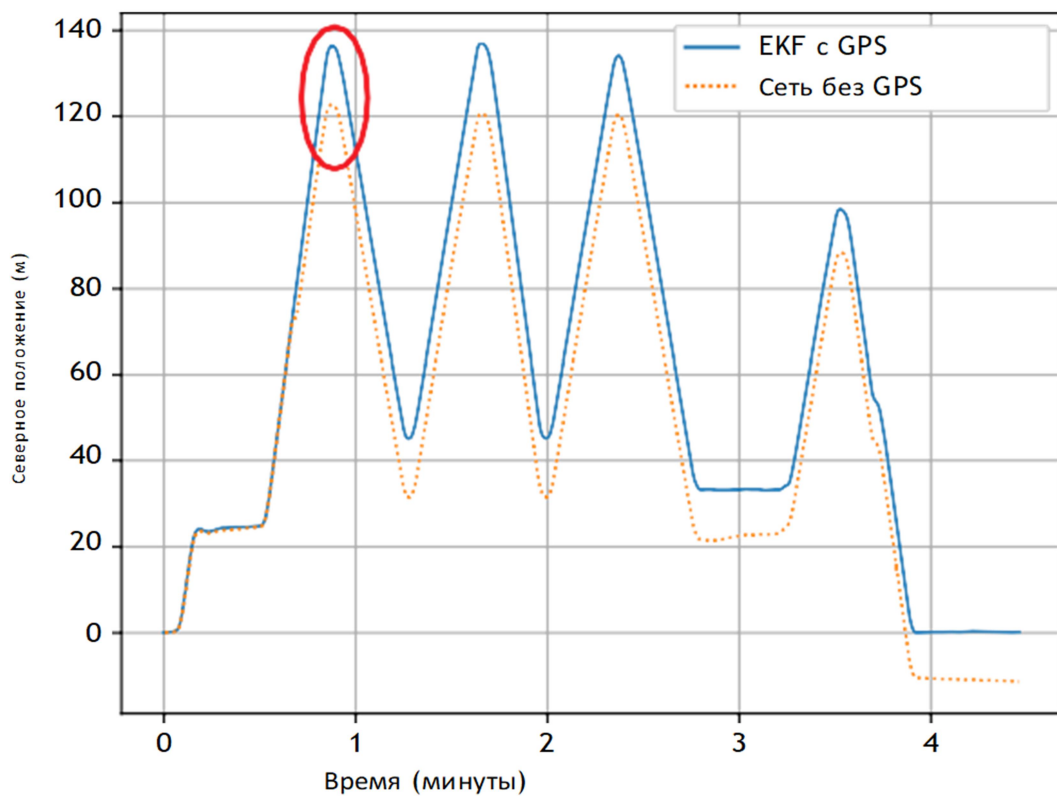


Рисунок 2.2 – Неверно предсказанное изменение приводит к смещению при восстановлении пути состояния, но оно не увеличивается

2.2.4 Обрезка времени заземления

При настройке сети было обнаружено, что сеть “протекает”, то есть, когда истинные значения не меняются, сеть медленно дрейфует. Но когда истинные значения меняются, сетевые прогнозы корректно следуют за изменениями. Это можно увидеть на рисунке 2.3. Сеть узнала об этом поведении из основных значений истинности. Оказывается, EKF2 оценивает дрейф, когда беспилотник находится на земле, перед взлетом и после посадки, даже при наличии хорошего сигнала GPS.

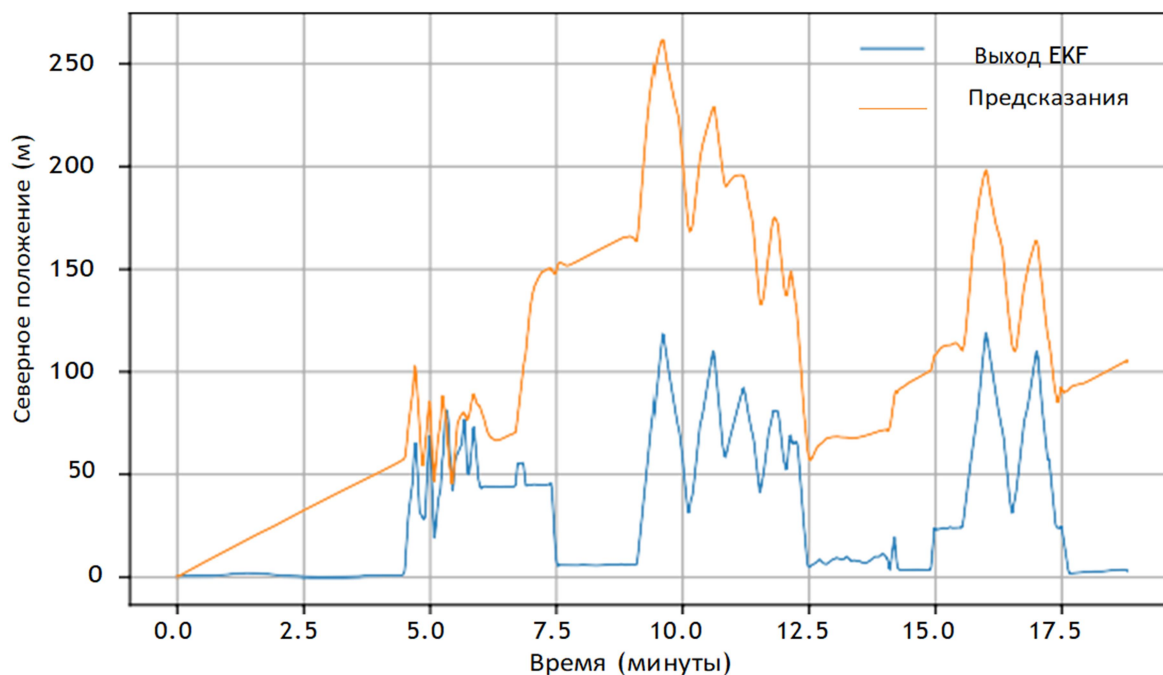


Рисунок 2.3 – Прогнозы сети изменяются, когда истинные значения не меняются

На рисунке 2.4 приведен пример тридцатиминутного журнала, из которого около десяти минут являются ошибочными основными значениями истинности. Такое поведение существует на большинстве логов, поэтому сети трудно не узнать об этом. Это поведение можно исправить, сократив время ожидания из журналов перед отправкой их в сеть.

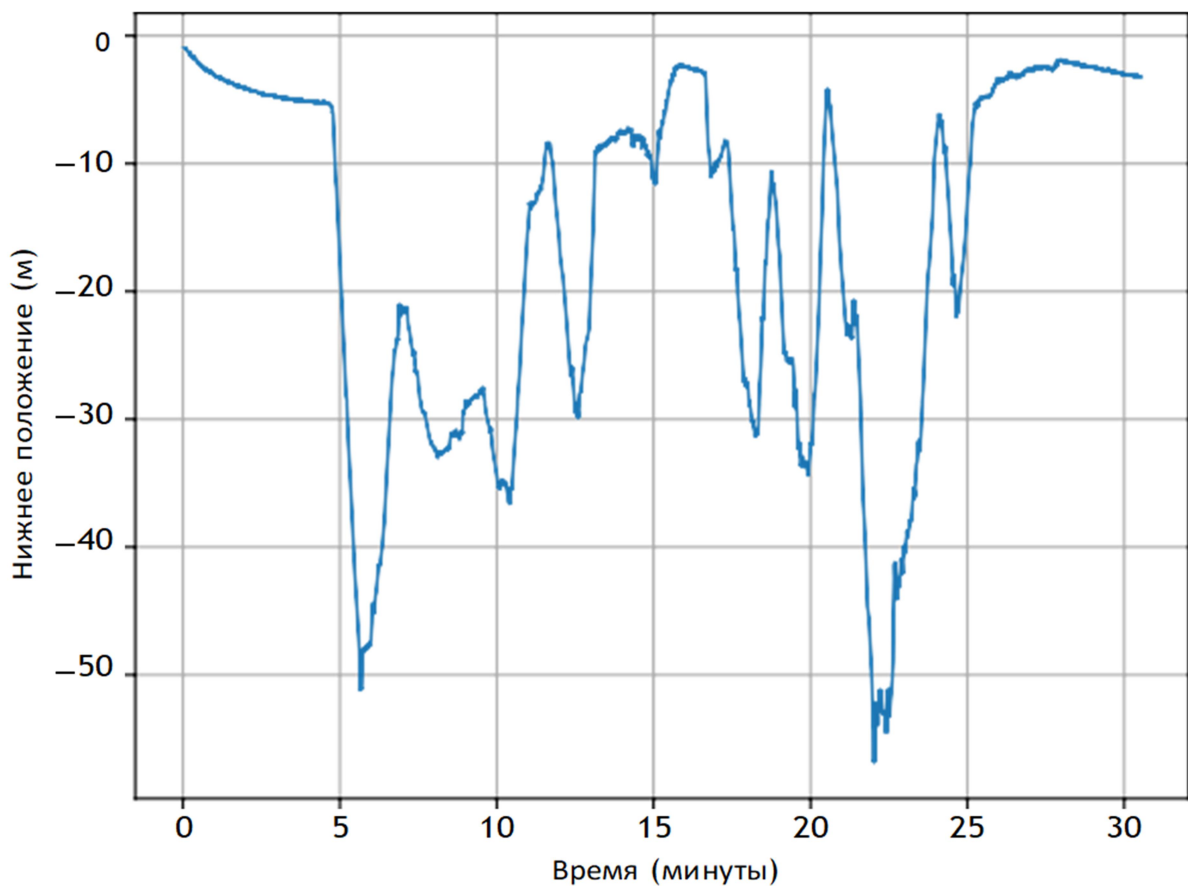


Рисунок 2.4 – EKF оценивает смещение, когда беспилотник находится на земле

2.2.5 Ручная очистка набора данных

Дальнейшая ручная очистка была применена для устранения других проблем в наборе данных. Например, некоторые записи, в которых беспилотник даже не взлетал, могут длиться до 20 минут. Другие журналы, собранные в случае неправильной конфигурации EKF или сбоя оборудования, также были удалены. В результате такой интенсивной очистки количество журналов сократилось. На рисунке 2.5 показаны журналы, имеющиеся в базе данных, с временем отчета около 21 минуты. После обрезки времени приземления было обнаружено, что весь журнал поврежден, и фактический взлет не состоялся.

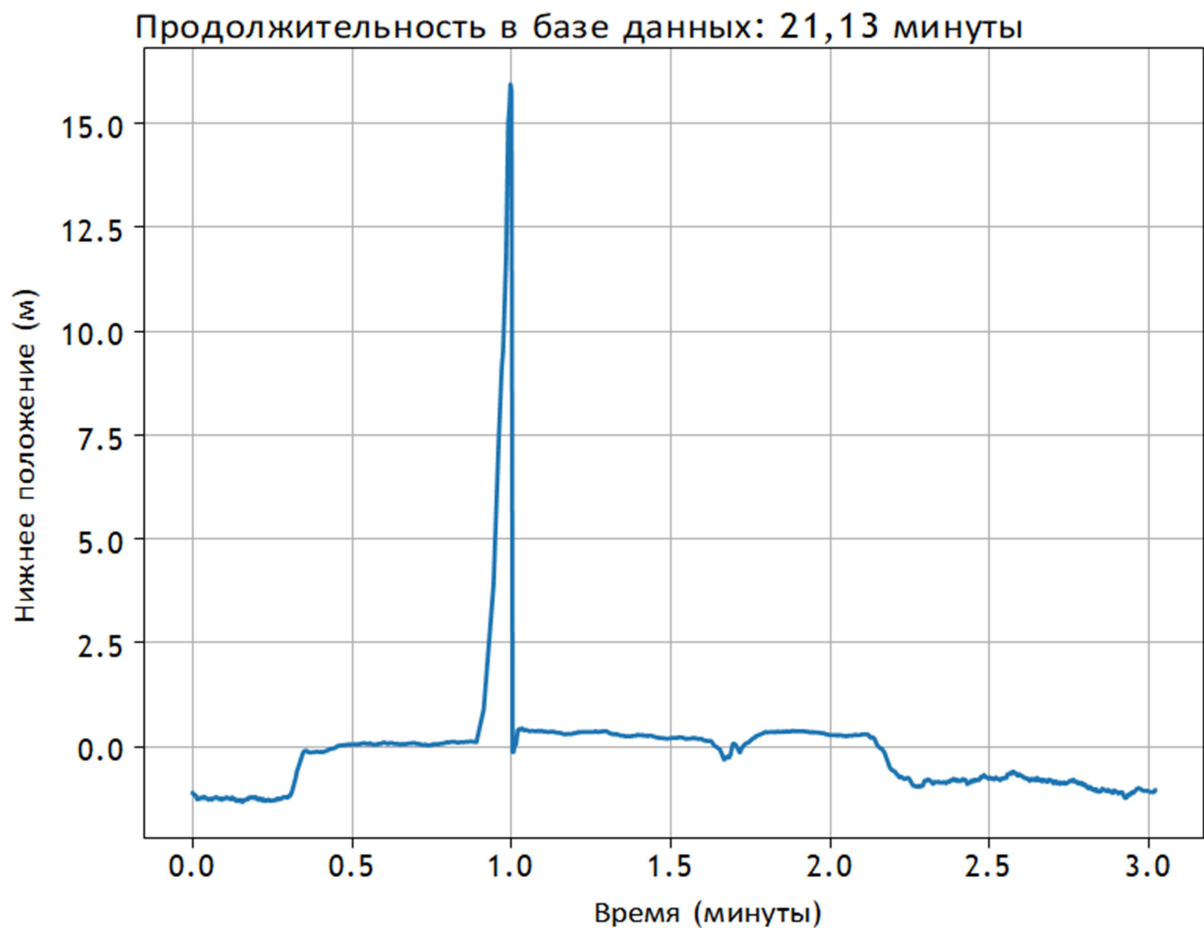


Рисунок 2.5 – журнал без фактического взлета

2.3 Проектирование сети и настройка гиперпараметров

2.3.1 Общие требования

Проектирование нейронной сети - это итеративный процесс. Необходимо выбрать множество проектных параметров, определяющих форму и характеристики сети. Другие проектные решения связаны с тем, как обучается сеть, то есть как обновляются веса. Эти решения приводят к выбору правильных гиперпараметров. В этом разделе описано, как был выбран каждый конструктивный параметр и гиперпараметр. В приложении Г показана общая архитектура сети. Количество слоев, их типы и количество узлов в каждом слое - вот некоторые из дизайнерских решений. Чтобы принять решение о гиперпараметрах, необходим индекс производительности.

Если проект допускает единственную ошибку в начале данного полета, то в течение всей продолжительности полета сохраняется постоянное смещение между истинной и расчетной траекторией. Во время обучения

можно рассматривать только первую перспективу через величину потерь (издержек). Второй метод оценки требует постобработки после обучения сети. Трехмерная траектория каждого проверочного полета сравнивается с истинной трехмерной траекторией.

2.3.2 Функция потерь

Функция потерь – это функция, которая принимает два значения – истинное (y) и прогнозируемое ($\hat{y}$), и возвращает оценку сравнения, насколько прогнозируемое значение близко к истинному. Чем меньше эта оценка, тем лучше, и наоборот — большие значения оценки указывают на то, что модель нуждается в доработке. Основная задача в процессе обучения нейронной сети, сводить к минимуму разницу между истинным и прогнозируемым значением в готовой модели.

Решаемые нейронными сетями задачи сводятся к двум глобальным категориям — регрессия и классификация. Например, компоненты скорости ограничены двигательными и конструктивными возможностями транспортного средства. Они значительно меньше, чем компоненты позиционирования, которые могут увеличиваться настолько, насколько позволяет канал передачи данных дрона с наземной станцией. Это приводит к большим ошибкам в скорости, рис. 15, даже если ошибки в местоположении невелики, рис. 16.

Возвращает среднее всех абсолютных значений разницы между фактическим и прогнозируемым. Данная функция, наоборот, более предпочтительна для работы с большим количеством выбросов. Так как функция работает с абсолютом (модулем), то все отрицательные значения становятся положительными, и соответственно, будут налагаться меньшие штрафы за большую разницу. Уравнение 1 представляет собой стандартное значение MAE, Уравнение 2 является взвешенным, где ошибка сигнала умножается на вес сигнала. Вес для каждого сигнала является величиной, обратной его среднему значению в обучающем наборе.

$$\text{MAE} = \frac{\sum_{i=1}^n |\hat{y}_i - y_i|}{n} = \frac{\sum_{i=1}^n |e_i|}{n} \quad (1)$$

$$\text{MAE}_{\text{weighted}} = \frac{\sum_{i=1}^n |\hat{y}_i - y_i| w_i}{n} = \frac{\sum_{i=1}^n |e_i| w_i}{n} \quad (2)$$

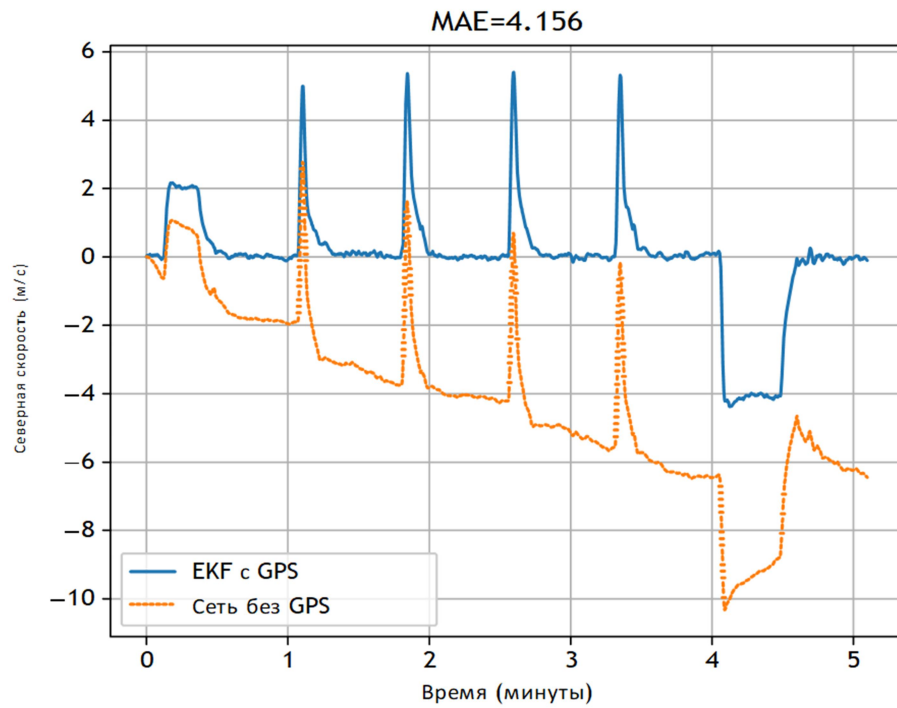


Рисунок 2.6 – Из-за меньших величин погрешностям скорости уделяется меньше внимания

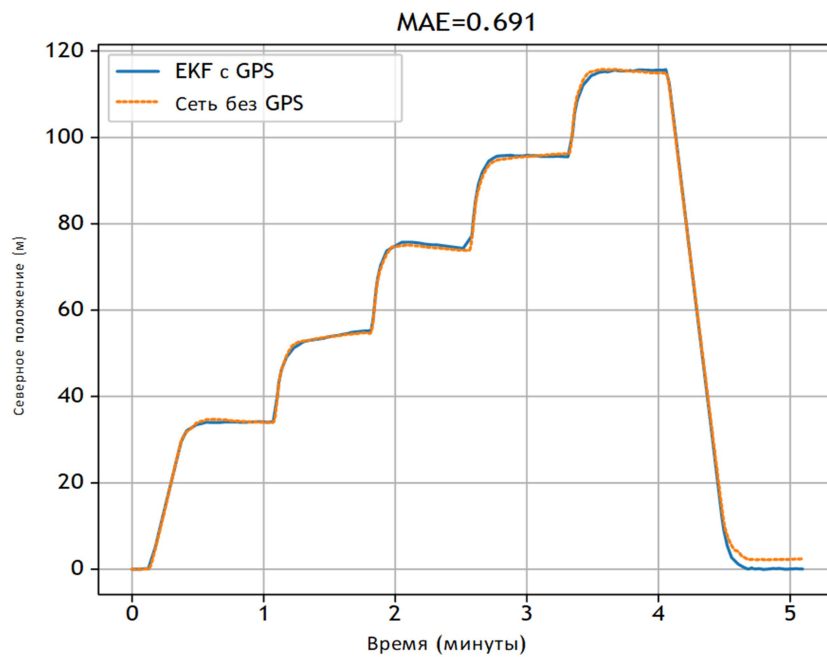


Рисунок 2.7 – Точность положения

2.4 Настройка гиперпараметров

2.4.1 Количество Узлов

Наилучшая производительность была достигнута при использовании четырех повторяющихся слоев по 200 узлов каждый, а затем одного плотного слоя из 6 узлов, который изменял форму данных в соответствии с формой меток.

2.4.2 Тип повторяющихся ячеек и функции активации

В ячейке LSTM применяются две активации: одна для новых входов, а другая для повторяющихся входов. Повторная активация должна быть сигмовидной, чтобы работало ускоренное обучение на графическом процессоре.

Таблица 2.2 – Влияние размера окна

Размер окна	Медиана максимума ошибки положения в проверочный набор (метры)	Время обучения, 100 эпохи (часы)
50	72,4	1,94
100	52,78	4,94
150	35,59	7,25
200	34,26	9.21

2.4.3 Размер окна

2.4.3 Увеличение размера окна повышает точность, но значительно увеличивает как время обучения, так и время вывода. Слишком большие окна могут снизить точность, поскольку контекст большего размера, чем необходимо, приводит к путанице в сети. В таблице 2.2 показано влияние увеличения размера окна как на точность, так и на время обучения.

2.4.4 Скорость обучения

2.4.4 Были протестированы скорости обучения от 0,00001 до 0,1. В начальные периоды, когда веса далеки от своих оптимальных значений, можно использовать высокую скорость обучения. Когда потери приближаются к своему минимуму, скорость обучения следует снизить, чтобы предотвратить превышение. Таблице 2.3 была использована в качестве графика темпов обучения.

2.4.5 Размер партии

Количество обучающих выборок (окон), передаваемых в сеть на каждом шаге оптимизации, называется размером пакета. Использование

пакетной обработки может привести к более быстрой сходимости, так как веса обновляются чаще. Выбранный размер пакета равен 1024, поскольку он обеспечивает наиболее быстрое обучение и некоторую регуляризацию.

Таблица 2.3 – Расписание скорости обучения

Эпохи	Скорость
0 - 50	0.005.
0 - 50	0.0025.
101 - ...	0.001.

2.4.6 Регуляризация

Наиболее распространенным методом регуляризации является отсев, который деактивирует скрытые узлы случайным образом при распространении каждого входного вектора во время обучения; выполняется две активации, поэтому применяется два отсева, один для входной и один для повторных активаций. В более ранних испытаниях отсев давал очень хорошую регуляризацию при использовании небольшого количества бортовых журналов. К сожалению, повторяющийся сбой предотвращает ускорение графического процессора, поэтому его нельзя использовать с большими наборами данных.

Таблица 2.4 – Сетевая архитектура и гиперпараметры

Сенсор	Модель
Количество повторяющихся слоев	4
Количество узлов в повторяющейся ячейке	200
Количество плотных слоев	1
Количество узлов в плотном слое	6
Скорость затухания	0,9, 0,999
Количество эпох	100
График скорости обучения	0,005, 0,0025, 0,001
Размер окна	200 шагов = 40 секунды
Размер партии	1024 примера

2.4.7 Время обучения и количество эпох

Эпоха - это один проход по всему набору данных, т.е. обработка всех пакетов. Более длительное время обучения повышает точность обучения, но может снизить точность валидации переобучения. Были протестированы

различные количества эпох от 100 до 1500, но точность не сильно увеличилась после 200 эпох.

2.4.8 Окончательный дизайн

Дизайн сети и использованные гиперпараметры представлены в таблице 2.4.

3 Обучение сети

3.1 Результаты и обсуждение

Показатель MPE, определенный в разделе "В", является достаточным для сравнения различных конструкций, поскольку оба полета в тренировочном и тестовом наборах инвариантны. Однако при сравнении точности конструкции в двух разных полетах одного MPE недостаточно, поскольку разные полеты имеют разную продолжительность и расстояние в пути. Традиционно MPE оцениваются с точки зрения временного увеличения ошибки позиционирования, поскольку основная ошибка оценки в MPE накапливается пропорционально времени от начала позиционирования. В дальнейшем MPE делится на время полета, и для описания используется нормированная по времени максимальная ошибка позиционирования (TN-MPE), выраженная в метрах в минуту (м/мин).

3.2 Производительность набора для тренировки

Эффективность обучения обычно проверяется, чтобы убедиться, что сеть, по крайней мере, соответствует данным обучения. Не следует вносить никаких улучшений в производительность проверки до тех пор, пока сеть в достаточной степени не будет соответствовать обучающим данным. Однако эффективность обучения не является достаточным показателем, и не следует ожидать, что сеть будет работать аналогичным образом с более свежими данными. В таблице 3.1 приведены результаты для обучающего набора из 465 полетов.

Таблица 3.1 – Производительность в обучающем наборе данных

	MPE (м)	TN-MPE (м/мин)	MVE (м/с)
Среднее	16,78	2,98	3,78
Медиана	7.23	1,39	2,26
Лучший	1.13	0,45	0,57
Худший	1666,31	262,69	168.02

3.3 Производительность набора для проверки

Эффективность проверки отражает то, как система работает с новыми рейсами, не использовавшимися во время обучения. Набор проверочных данных состоит из 83 рейсов общей продолжительностью около 8 часов. В таблице 3.2 приведены сводные данные о производительности набора для

проверки. Среднее значение и медиана рассчитываются исходя из максимальных ошибок, достигнутых в каждом полете.

Таблица 3.2 – Производительность набора данных проверки

	MPE (м)	TN-MPE (м/мин)	MVE (м/с)
Среднее	85,79	17,79	6.4
Медиана	34,26	6	4.15
Лучший	2,66	0,61	0,76
Худший	871,54	142.08	67,65

Некоторые полеты начались в воздухе из-за сокращения наземного времени, как описано в разделе Б.4., и взлетная часть приходится на первое окно. Как указано в таблице 2.4, размер окна составляет 200 шагов. Это означает, что сеть делает свой первый прогноз через 40 секунд. К этому времени беспилотник, возможно, уже достиг заданной высоты полета. Несмотря на то, что траектория на рисунке 2.8 претерпела некоторые резкие изменения направления, сеть все еще могла правильно предсказать эти маневры. Одна из причин заключается в том, что такая схема полета является распространенной в наборе данных, поэтому в сети было достаточно данных, чтобы связать поведение соответствующего датчика с результирующими изменениями положения.

На рисунке 3.1 показан еще один проверочный полет с различными маневрами. Синяя стрелка указывает на внезапное изменение скорости разворота; в этот момент делается ошибочный прогноз, приводящий к смещению, которое сохраняется до конца полета. На рисунках 3.2, 3.3 и 3. показаны еще три проверочных полета. Несмотря на самую короткую продолжительность полета, полет на рисунке 3.2 имеет самый высокий MPE среди трех и более чем в два раза больше TN-MPE, чем любой из них. Это связано с тем, что предсказания на рис. 3.2 содержат множество ошибок ближе к началу.

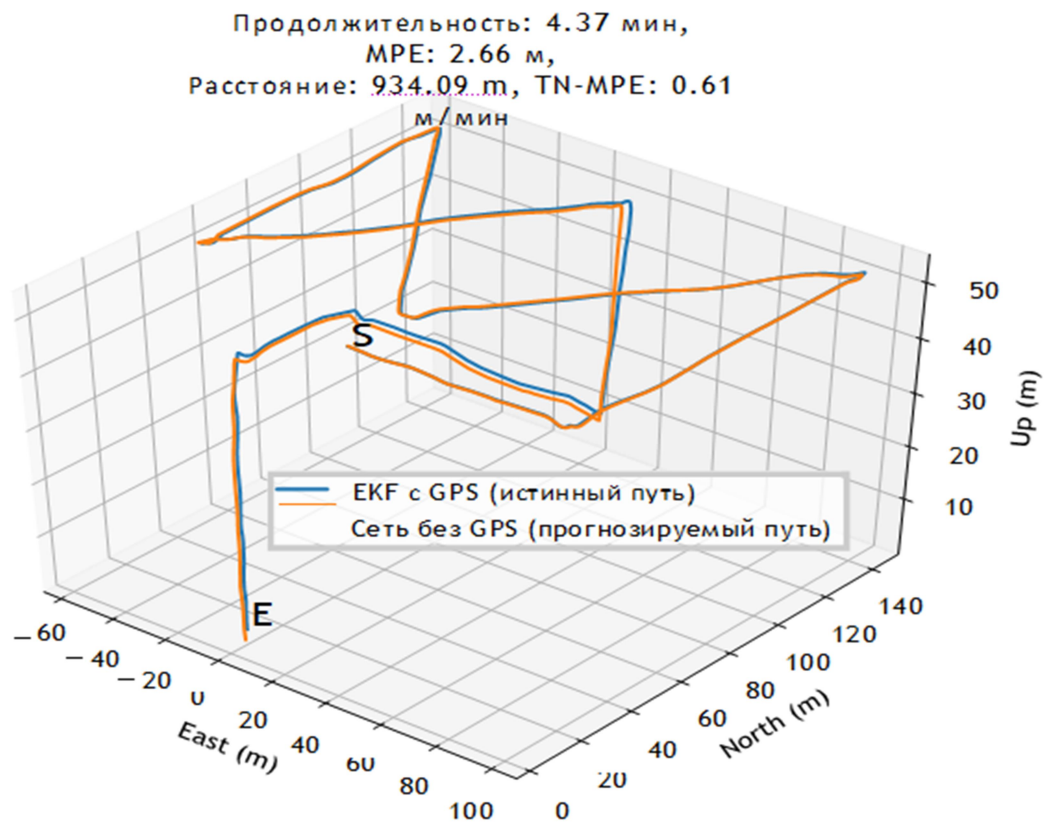


Рисунок 3.1 – Траектория наилучшего проверочного полета

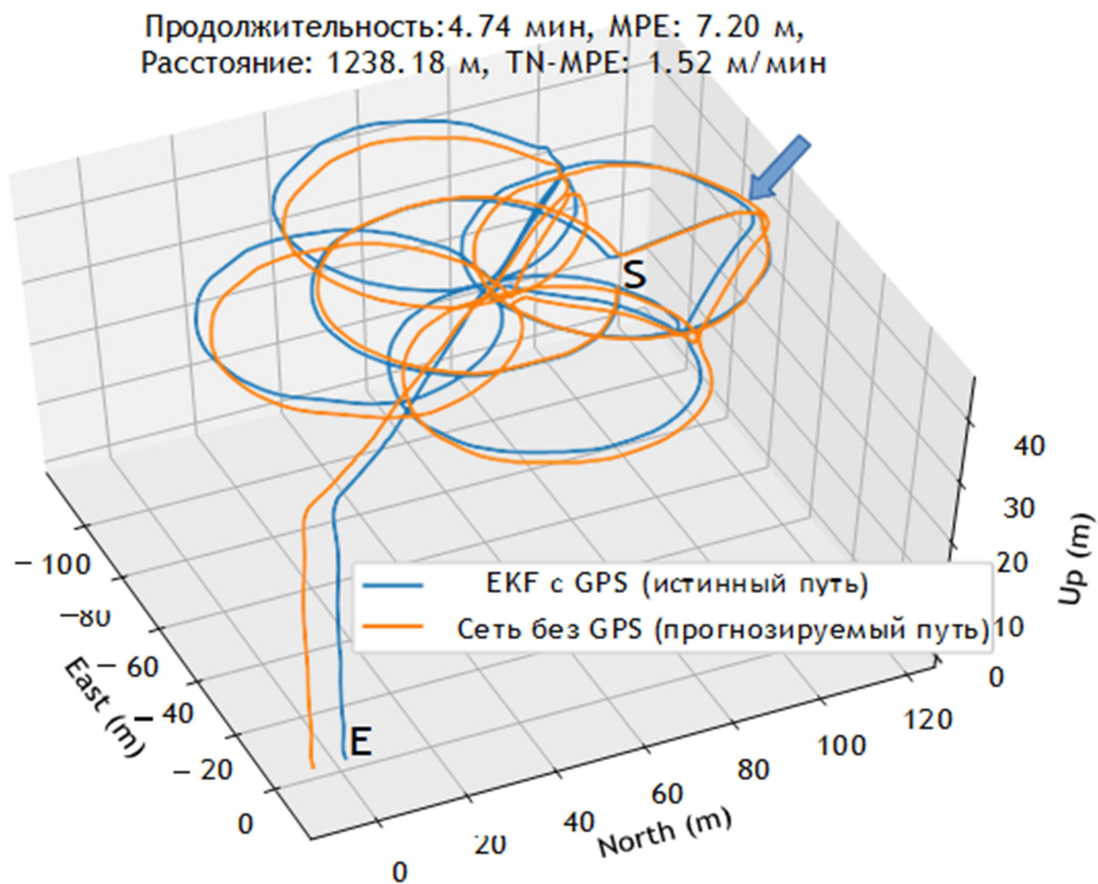


Рисунок 3.2 – Валидационная траектория полета -2

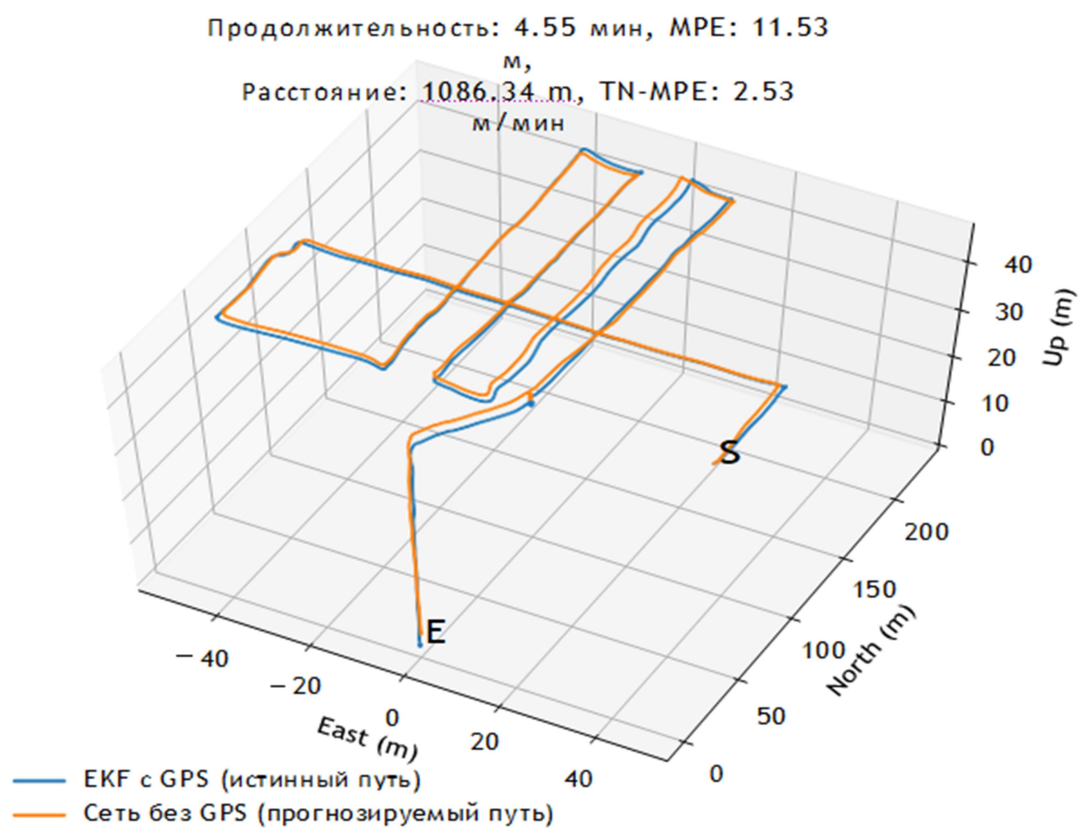


Рисунок 3.3 – Валидационная траектория полета -1

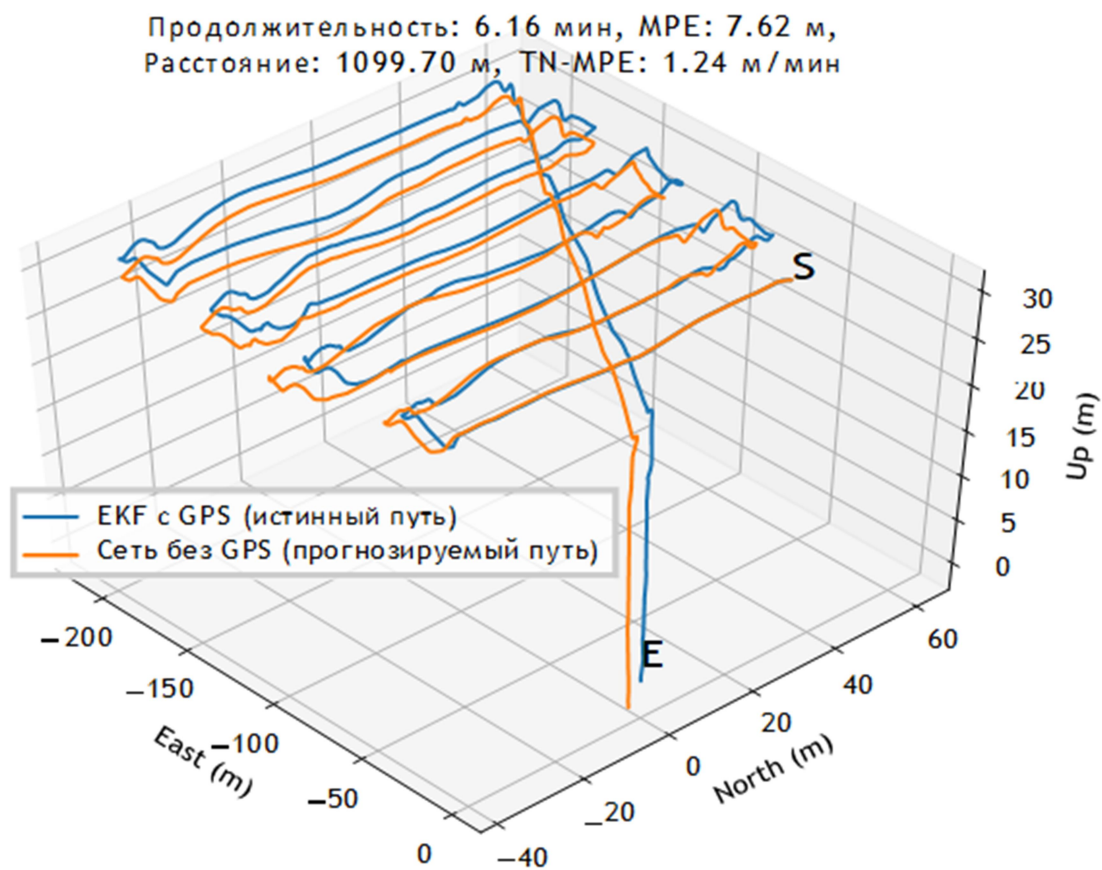


Рисунок 3.4 – Валидационная траектория полета -3

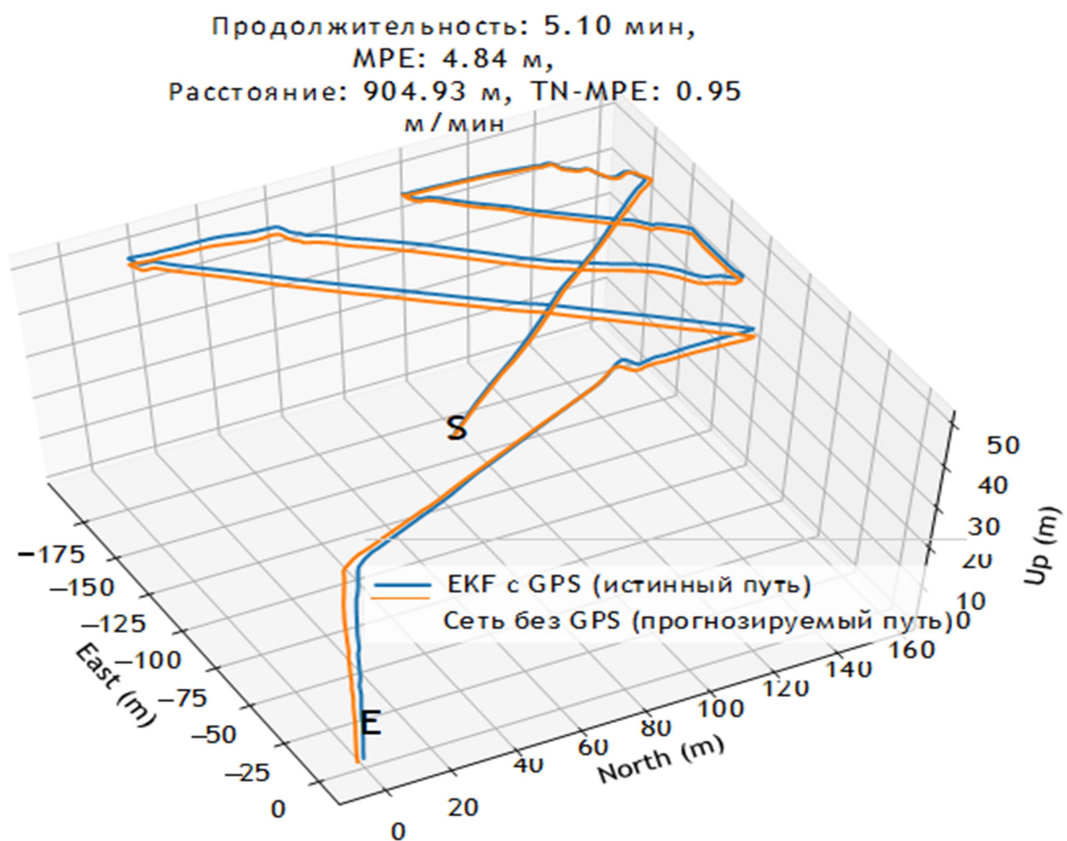


Рисунок 3.5 – Валидационная траектория полета -4

3.3.1

4 Тестирование

4.1 Предсказания скорости

Сеть, описанная в таблице 2.4, независимо предсказывает как разности положения, так и скорости (приращения), поэтому ошибки в некотором направлении скорости не обязательно отражаются на ошибках положения в том же направлении. Другой метод вычисления скорости заключается в разделении разностей положений, вычисленных сетью, на временной шаг. Поскольку временной шаг фиксирован равным 0,2 секунды, любую прогнозируемую разницу в местоположении можно разделить на 0,2, чтобы получить скорость в том же направлении. Это не только уменьшает размер сети и объем вычислений, но и повышает точность вычисляемой скорости. На рис. 4.1, рис. 4.2 и рис. 4.3 сравниваются два метода для трех составляющих скорости в проверочном полете.

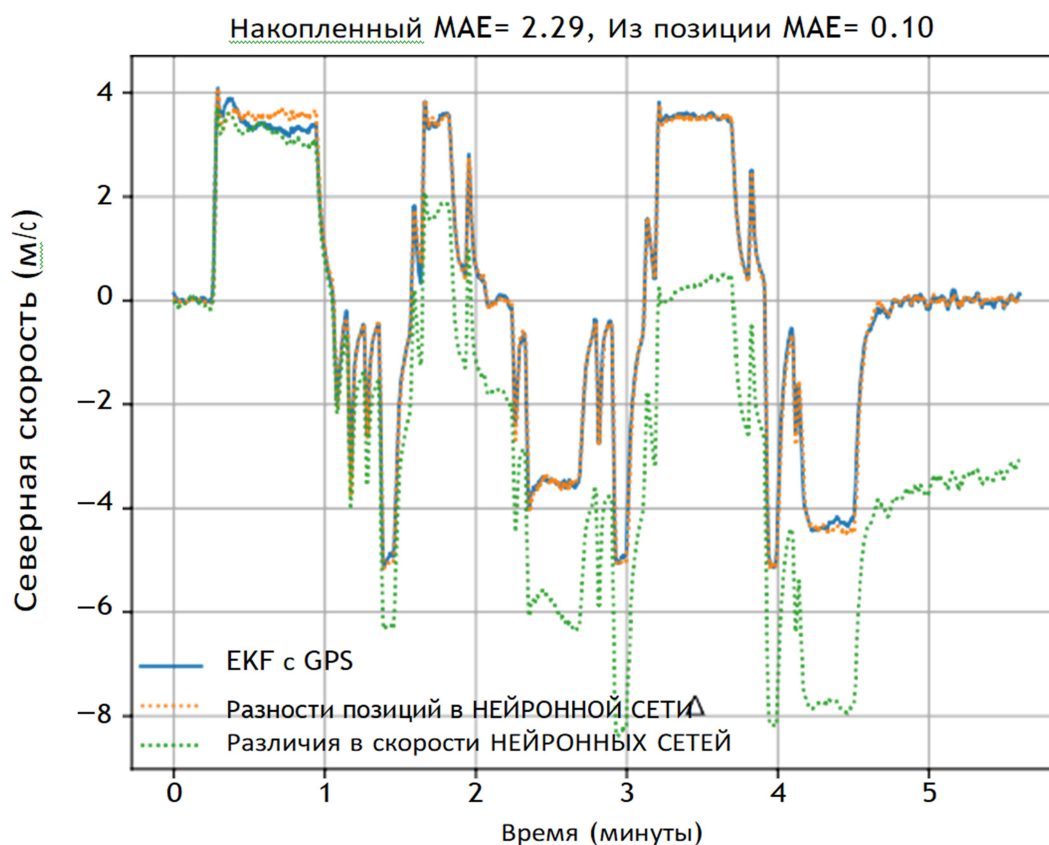


Рисунок 4.1 – Составляющая скорости на север для проверочного полета, прогнозируемая сетью, по сравнению с вычисленной на основе различий в местоположении

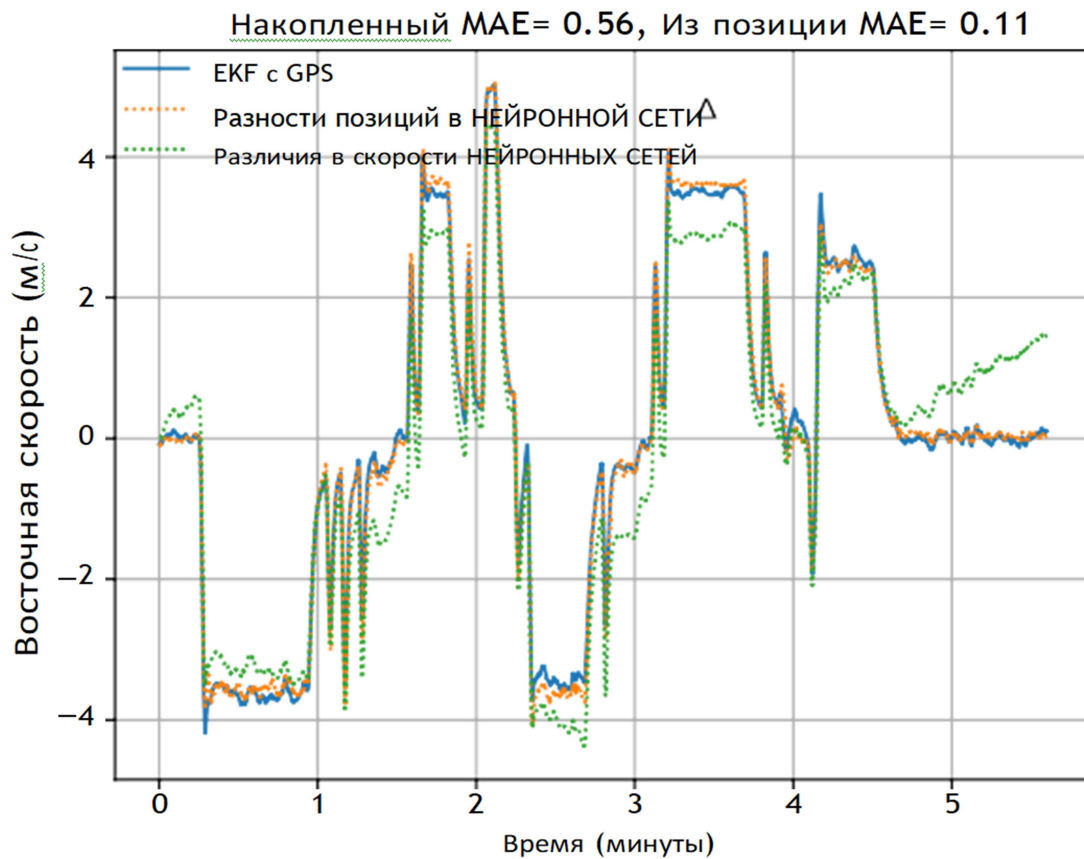


Рисунок 4.2 – Восточная составляющая скорости проверочного полета, прогнозируемая сетью, по сравнению с вычисленной на основе различий в местоположении

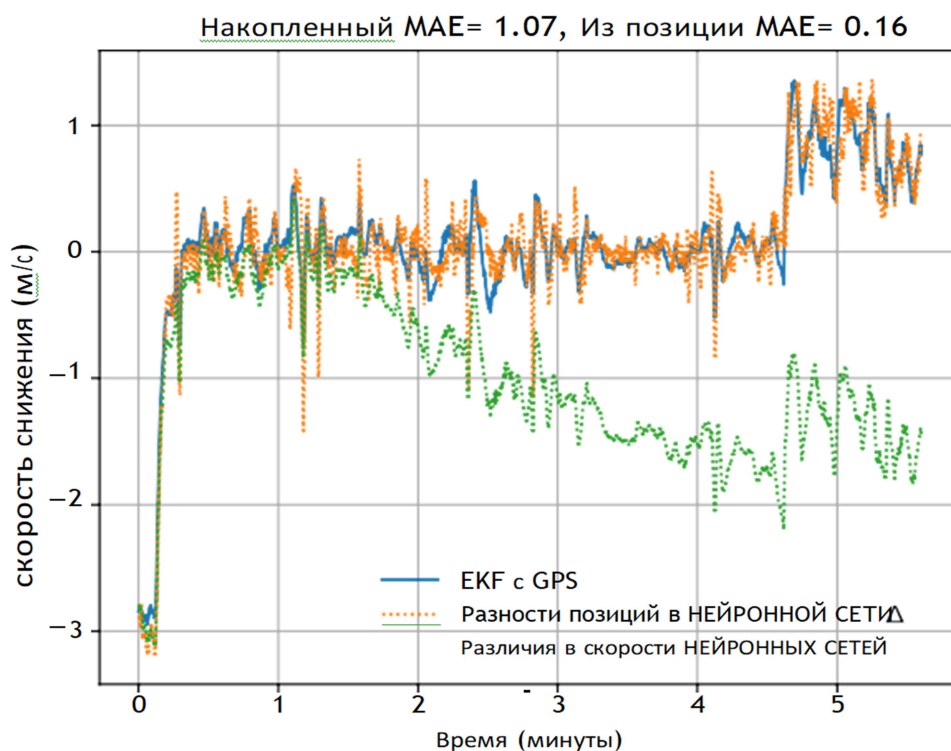


Рисунок 4.3 – Составляющая скорости снижения при проверочном полете, прогнозируемая сетью, по сравнению с рассчитанной на основе различий в местоположении

4.2 Предсказания отношения

PX4 EKF2 оценивает отношение, выраженное в кватернионах. Эти оценки ориентации не расходятся при потере сигнала GPS, поскольку гироскоп и акселерометр могут надежно предсказывать углы крена и тангажа, а магнитометр помогает корректировать угол рыскания.. На рисунках (4.4-4.7) показано положение, предсказанное сетью и оцененное EKF при отсутствии GPS. Ошибки EKF здесь являются результатом численных сложностей, вызванных значительными расхождениями в оценках положения и скорости. В коде PX4 это устраняется путем передачи измерений нулевого положения и скорости в EKF при потере GPS-фиксации, чтобы сохранить числовую стабильность других оценочных величин, таких как ориентация. Эти разумные оценки, полученные с помощью EKF без GPS, являются причиной того, что окончательный проект сети не предсказывает ориентацию. Добавление большего количества элементов к вектору меток усложняет сеть и требует больше времени как для обучения, так и для вывода. Требование большего количества данных от сети также отвлекает некоторое внимание от расчетов местоположения и скорости и снижает их производительность.

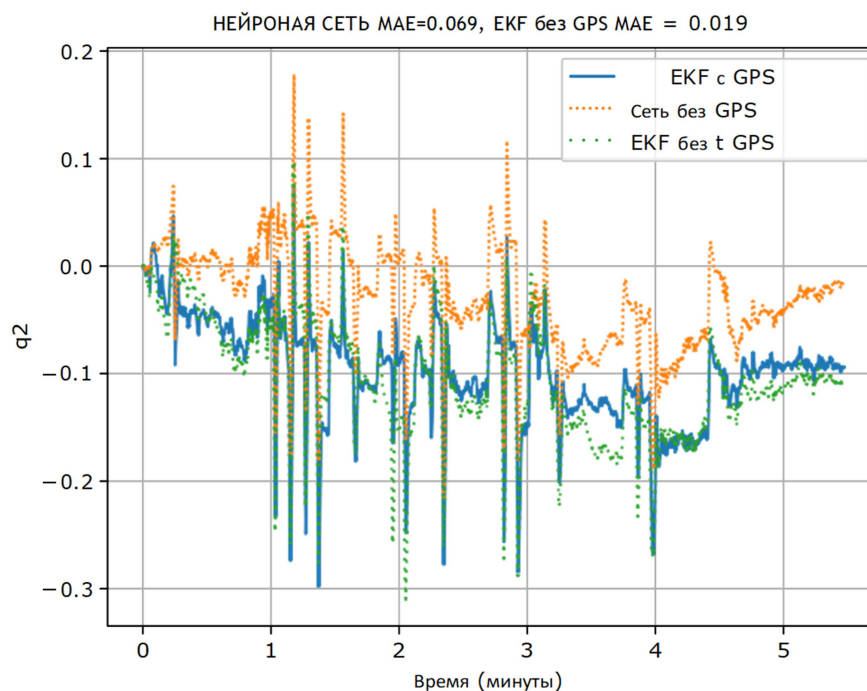


Рисунок 4.5 – Кватернионы отношения - q_2

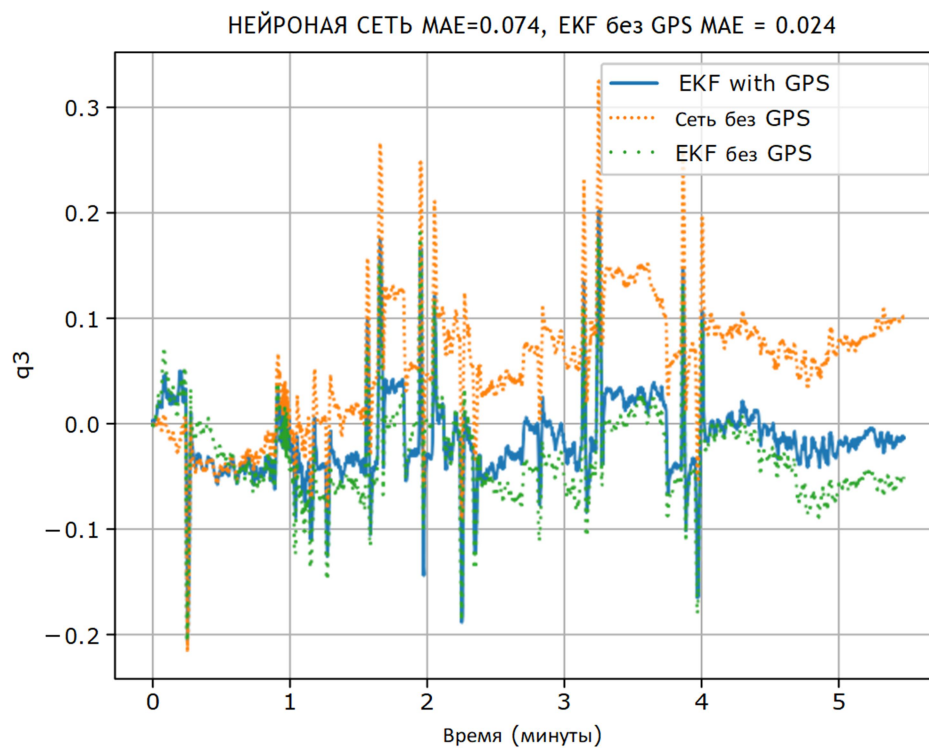


Рисунок 4.6 – Кватернионы отношения - q_3

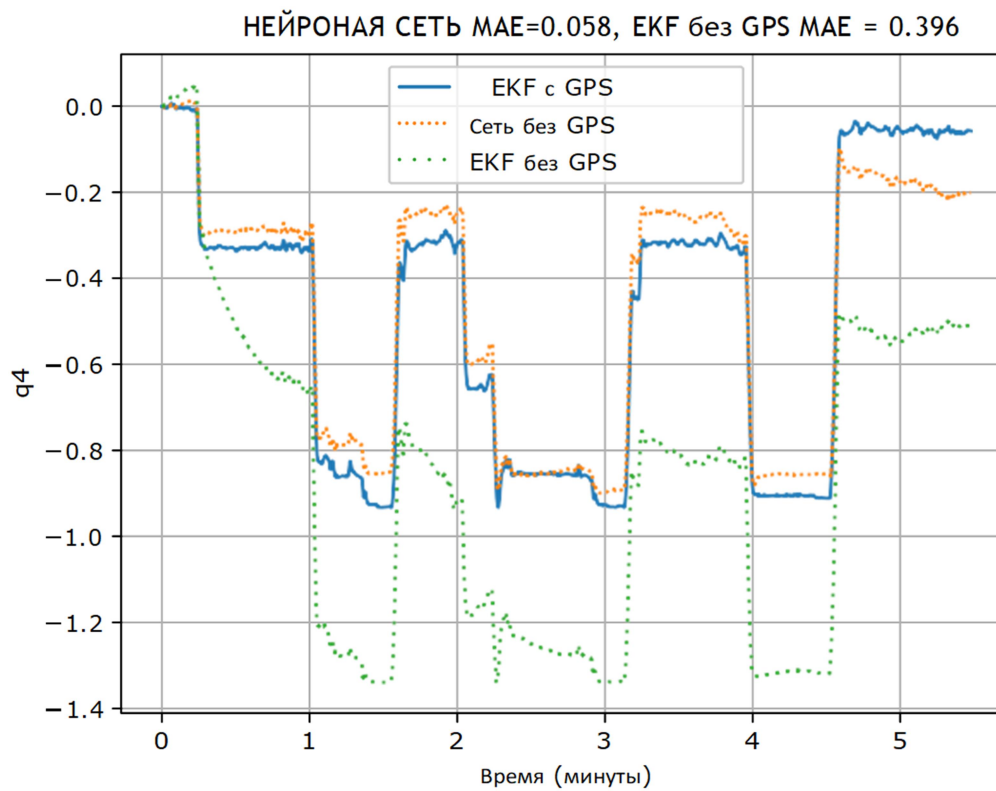


Рисунок 4.7 – Кватернионы отношения - q_4

ЗАКЛЮЧЕНИЕ

В работе была представлена нейронная сеть, способная предсказывать трехмерное местоположение и скорость дрона в отсутствие GPS, используя необработанные измерения ИИУ, барометра и магнитометра. Разработанная сеть была протестирована на Симуляции. При позиционировании были получены ошибки до 2,7. Хотя система может оценивать положение, EKF является более надежным для таких задач. Сеть настроена на выполнение выводов с частотой 5 Гц, что медленнее, чем EKF. Поэтому основная роль сети в целом - это перепозиционирование GPS, а не EKF. Сеть подходит для квадрокоптеров, работающих в режиме “автоматического” полета, поскольку большинство обучающих данных обладают этими характеристиками. Точность навигации повышается при использовании сети большего размера и окна большего размера. Но это происходит за счет времени как на обучение, так и на вывод. Средняя ошибка позиционирования на проверочном наборе в пять раз больше, чем на обучающем наборе. Это сигнализирует о необходимости надлежащей регуляризации, чтобы помочь сети лучше обобщать.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В пояснительной записке применяются следующие сокращения и обозначения:

EKF – Расширенный фильтра Калмана

GPS – Система глобального позиционирования

RNC – Рекуррентная нейронная сеть

MPE – Максимальная абсолютная разница в 3D-местоположении

ИНС – Инерциальная навигационная система

ИИУ – Инерциальное измерительного устройства

ИИУ – Инерциальное измерительного устройства

ИИУ – Инерциальное измерительного устройства

ИНС – Искусственная нейронная сеть

GRU – Gated Recurrent Unit, Управляемый рекуррентный блок

LSTM – ячейка долговременной кратковременной памяти

MPE – максимальная абсолютная разница

TN-MPE – Нормализованная по времени максимальная ошибка
определения местоположения

MAE – средняя абсолютная ошибка

MSE – среднеквадратичная ошибка

СибГУТИ – Сибирский государственный университет
телекоммуникаций и информатики

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

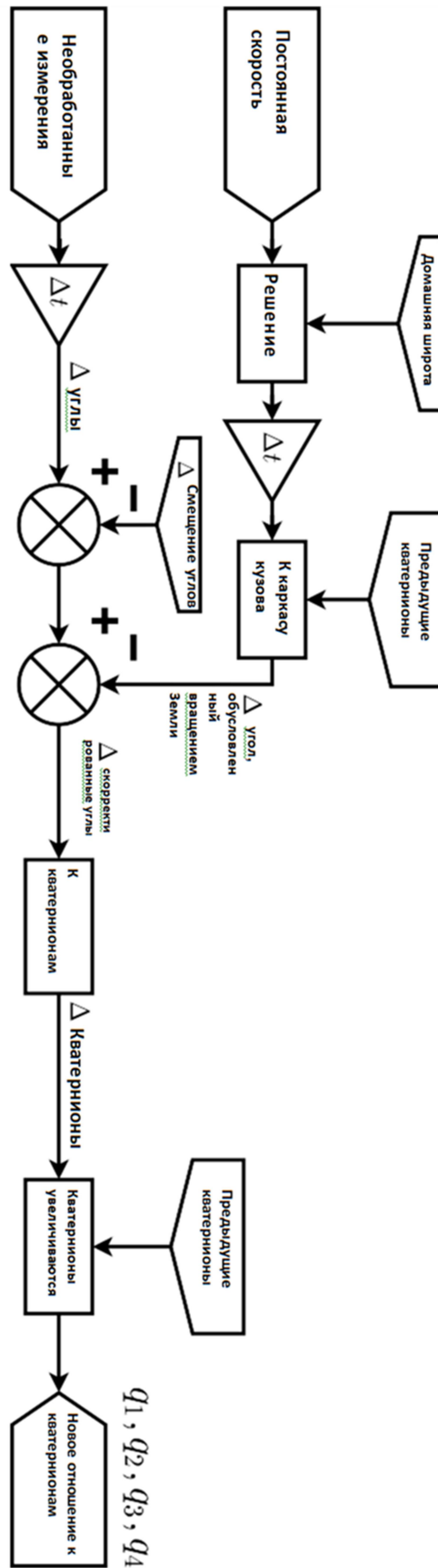
1. Павлова У.А., Ракитский А.А. Применение специализированных детерминированных конечных автоматов для прогнозирования временных рядов. Вестник СибГУТИ. 2022;(2):12-22. URL: <https://doi.org/10.55648/1998-6920-2022-16-2-12-22>
2. Efficient estimation of word representations in vector space / Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean // ICLR. — 2013.
3. Барский А. Б. Б26 Нейронные сети: распознавание, управление, принятие решений. — М.: Финансы и статистика, 2004. — 176 с: ил. — (Прикладные информационные технологии). ISBN 5-279-02757-X.
4. Harris, Z. Distributional structure / Zellig Harris // Word. — 1954.
5. Курс «Основы Нейронных Сетей » // NeuralNet.Info – URL: neuralnet.info/chapter/основы-инс (дата обращения: 12.05.19).
6. John Duchi Elad Hazan, Y. S. Adaptive subgradient methods for online learning and stochastic optimization / Yoram Singer John Duchi, Elad Hazan // JMLR. — 2011.
7. Kim, Y. Convolutional neural networks for sentence classification / Yoon Kim // IEMNLP. — 2014. — Sep. — 1746 -1751 p.
8. Курс «Neural Network Zoo» // The Asimov Institute – URL: <http://www.asimovinstitute.org/neural-network-zoo/> (дата обращения: 12.05.19)
9. Krizhevsky, A. Imagenet classification with deep convolutional neural networks / Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton // NIPS. — 2012. — 1106 -1114 p.
10. LeCun, X. Z. Y. Text understanding from scratch / Xiang Zhang Yann LeCun // Computer Science Department. — 2016.
11. McCulloch, W. S. A logical calculus of the ideas immanent in nervous activity / Warren S. McCulloch, Walter Pitts // Springer New York. — 1943.
12. Pennington, J. Glove: Global vectors for word representation / Jeffrey Pennington, Richard Socher, Christopher D // EMNLP. — 2014. — 1532 - 1543 p.

13. S, J. K. A statistical interpretation of term specificity and its application in retrieval / Jones K. S // Journal of Documentation. — 1972.

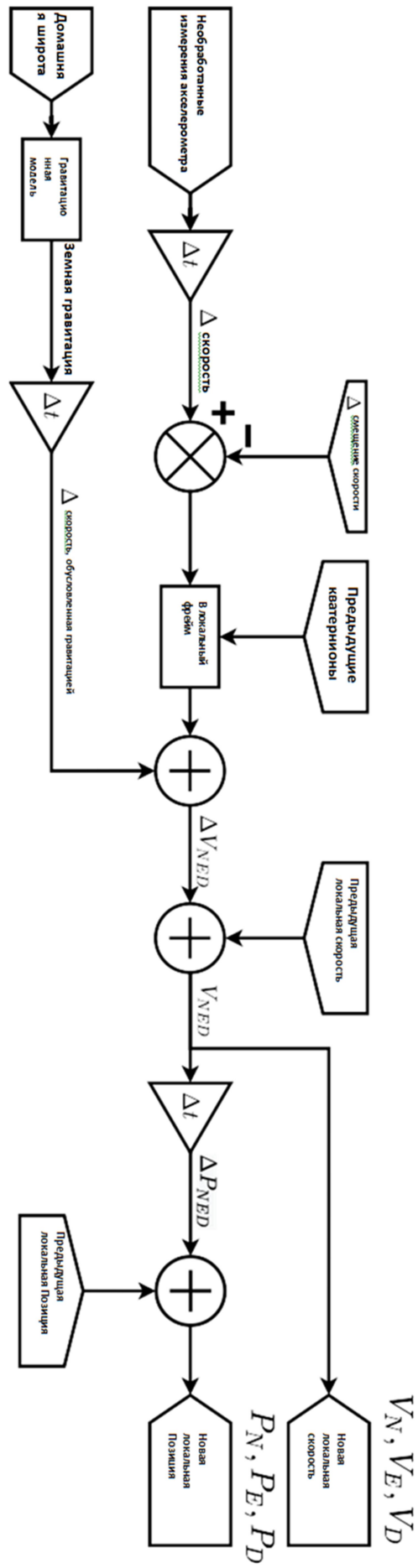
14. J. Turian et al. Word representations: A simple and general method for semi-supervised learning. Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, c. 384-394, 2010.

15. Goodfellow, I., Deep Learning (Adaptive Computation and Machine Learning series)/ I. Goodfellow, Y. Bengio, A. Courville - The MIT Press (November 18, 2016). – 800 c.

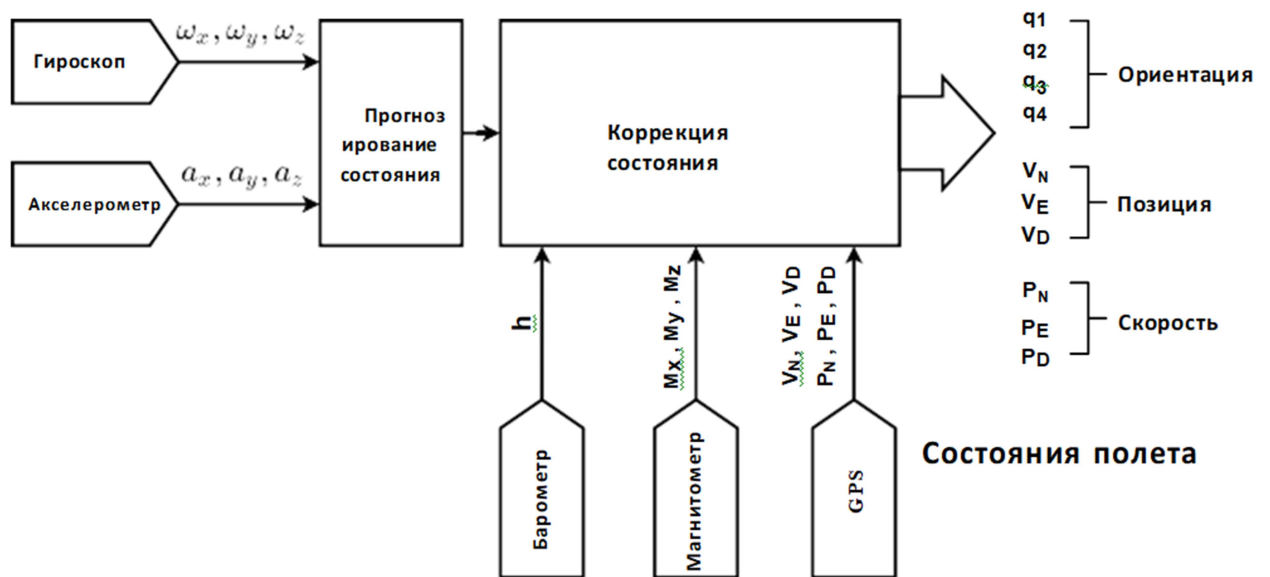
Предсказание ориентации по измерениям гироскопа на этапе прогнозирования ЕКФ 2



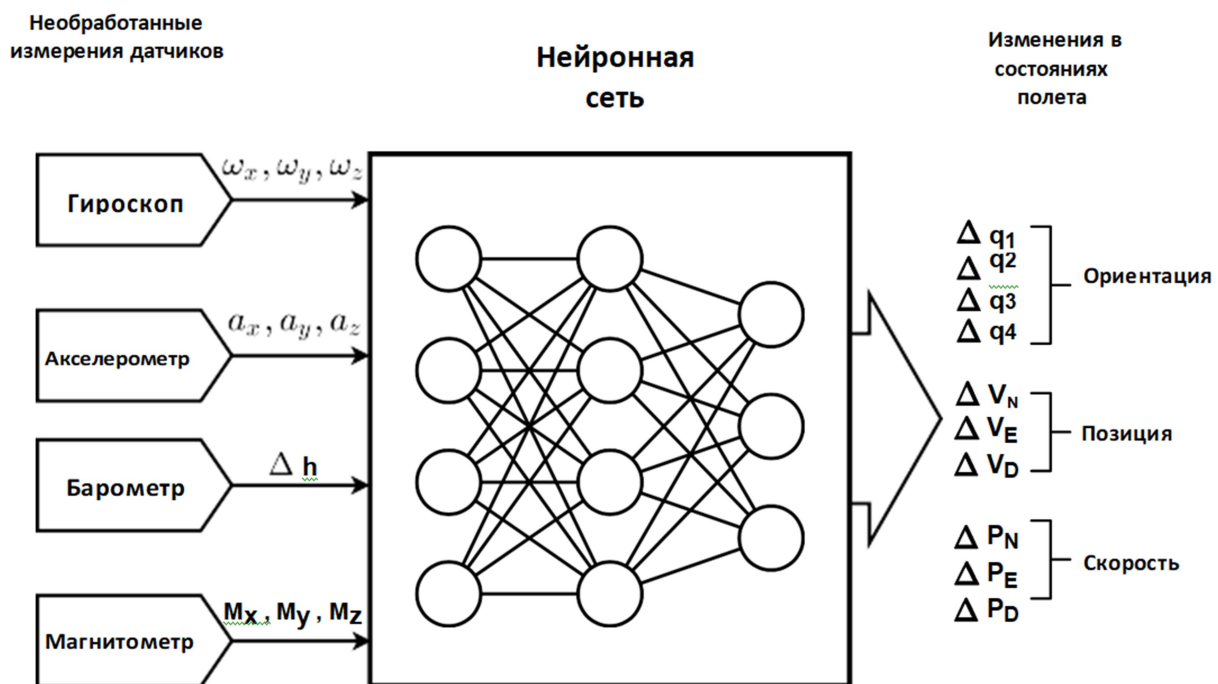
Предсказание положения по измерениям ориентации и акселерометра на этапе прогнозирования EKF2



Архитектура прогнозирования и коррекции ЕKF

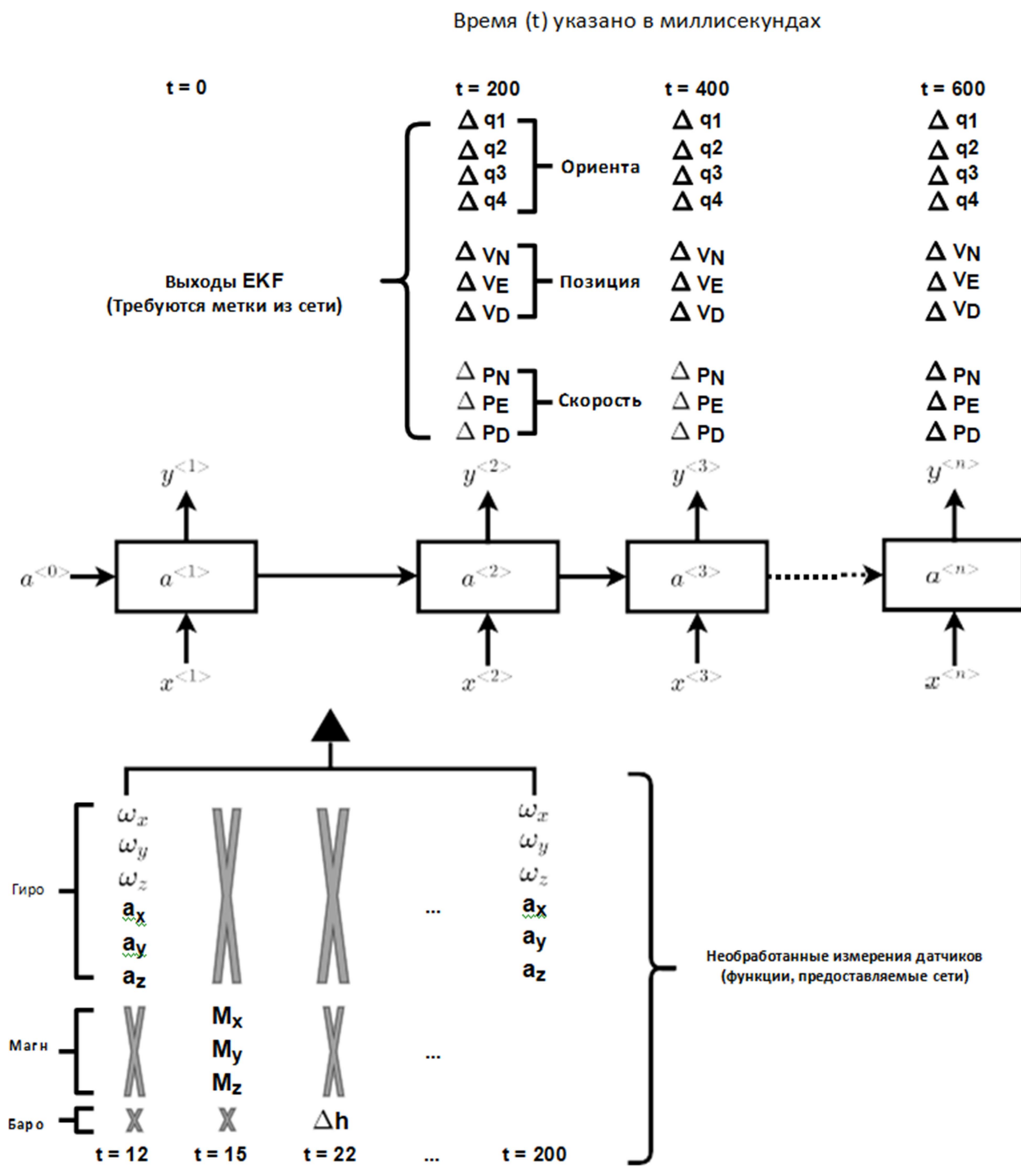


Предлагаемый оценщик на основе нейронных сетей



ПРИЛОЖЕНИЕ В

Объединение различных характеристик и меток, извлеченных из бортовых журналов



ПРИЛОЖЕНИЕ Г

Общая архитектура сети

